DATA STRUCUTRES AND ALGORITHMS

23CSE203

LAB MANUAL

Submitted by	
Name	Sowmya sree
Roll No	AV.SC.U4CSE25040
Year/Sem/Section	2nd year/Sem-3/CSE-A
Credits	05

Submitted to	
Name	Dr. RAJ KUMAR BATCHU
Department	School of Computing
Designation	Asst. Professor (SI Grade)

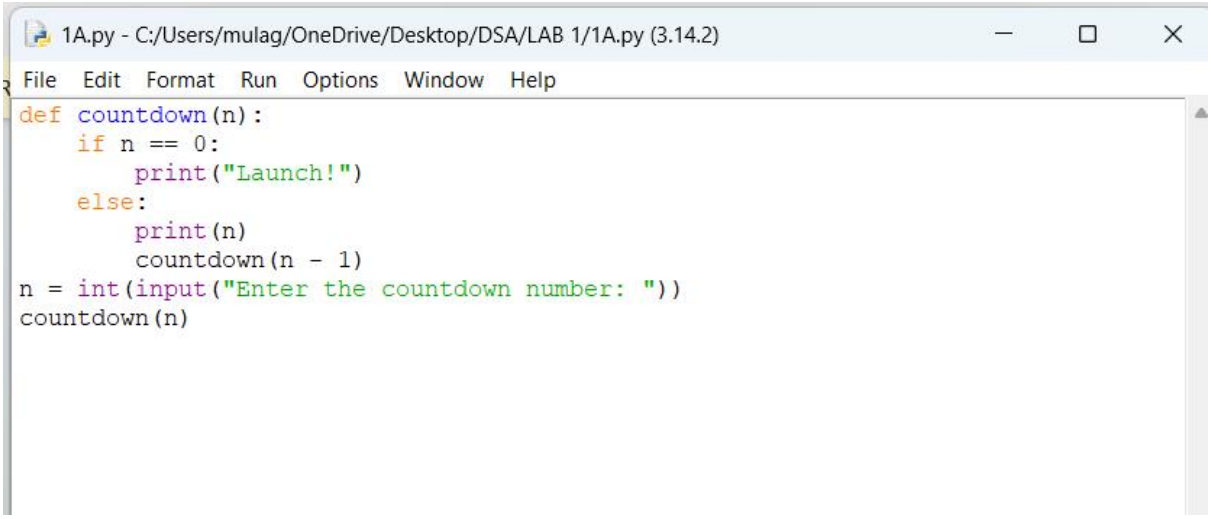
S.NO	TITLE	PAGE NO	REMARKS
1.	**WEEK 1:** 1a. Recursive countdown from n to Launch. 1b. Recursive calculation of compound growth factor. 1c. Recursive search for employee ID in list. 1d. Recursive factorial calculation for parcel arrangements. 1e. Recursive Fibonacci series generation.	3 - 9	
2.	**WEEK 2:** 2a. Python program to perform linear search on the given input. 2b. Python program to perform binary search on the given input. 2c. Verify by taking sorted array and unsorted array as an input.	9 - 14	
3.	**WEEK 3:** 3. Python program to perform bubble sort, insertion sort, selection sort, merge sort, quick sort on the given input.	14 - 20	
4.	**WEEK 4:** 4a. Python program to implement a menu-driven Singly Linked List (SLL) with the following operations: 1. Create a list 2. Insert at beginning/end/specific position 3. Delete by value/first/last node 4. Count and display nodes 5. Exit 4b. Python program to implement a menu-driven Doubly Linked List (DLL) with the following operations: 1. Create a list 2. Insert at beginning/end/specific position 3. Delete at beginning/end/specific position 4. Count and display nodes 5. Exit 4c. Python program to implement a menu-driven Circular Linked List (CLL) with the following operations: 1. Create a list 2. Insert at beginning/end/specific position 3. Delete at beginning/end/specific position 4. Count and display nodes 5. Display head and tail 6. Exit	20 - 34	
5.	**WEEK 5:** 5. Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.	34 – 39	
6.	**WEEK 6:** 6. Write a python program to implement queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach implement using array, linked list, circular linked list.	39 – 46	

7.	**WEEK 7:** 7. Write a python program to implement binary tree using an array and a linked list with all of its operation.	46 – 50	
8.	**WEEK 8:** 8. Write a python program to implement binary search trees with its operation.	50 - 53	
9.	**WEEK 9:** 9. Write a python program to implement AVL trees with its operation.	53 - 56	

WEEK – 1

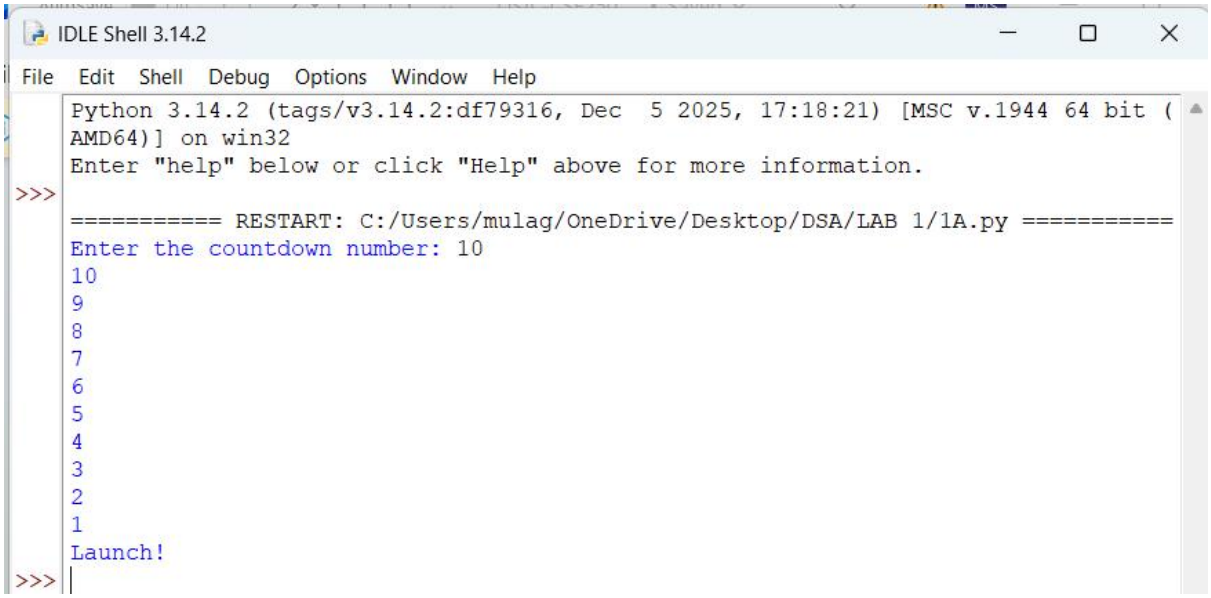
1a. AIM: A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursive. Write a recursive python program to display a countdown from n to 1, followed by "Launch".

PROGRAM:



OUTPUT:

Positive Case;



EXPLANATION:

- It takes a number as input from the user and stores it in **n**.
- It creates a recursive function named **countdown()** that prints the countdown.
- If **n** is greater than **0**, it prints the current value and calls **countdown(n - 1)** to continue the countdown.
- When **n** becomes **0** or less, it prints **"Rocket Launched"** and stops the recursion.
- Finally, it starts the recursive countdown by calling **countdown(n)**.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Syntax Error** Forgot to give the **":"** after the if, else, or function definition.	It was rectified by adding the missing **":"**.
2.	**Value Error** Entered a non-integer value instead of an integer.	It was rectified by entering only integer values as input.

1b. AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power), Where p = principal growth factor, n = number of years.

PROGRAM:

1B.py - C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1B.py (3.14.2)

File Edit Format Run Options Window Help

```
def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

p = int(input("Enter the principal growth factor (p): "))
n = int(input("Enter the number of years (n): "))
result = power(p, n)
print("Result =", result)
```

OUTPUT:

Positive Case;

IDLE Shell 3.14.2

File Edit Shell Debug Options Window Help

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1B.py ==========
Enter the principal growth factor (p): 15
Enter the number of years (n): 4
Result = 50625
>>>
```

Negative Case;

IDLE Shell 3.14.0

File Edit Shell Debug Options Window Help

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/compound interest.py ===
Enter the Principal Growth Factor:52.67
Enter the Number of Years:-3
Traceback (most recent call last):
  File "C:/Users/HP/OneDrive/Desktop/DSA CSE25045/compound interest.py", line 8, in <module>
    Output = power(p, n)
  File "C:/Users/HP/OneDrive/Desktop/DSA CSE25045/compound interest.py", line 7, in power
    return p * power(p, n - 1)
  File "C:/Users/HP/OneDrive/Desktop/DSA CSE25045/compound interest.py", line 7, in power
    return p * power(p, n - 1)
  File "C:/Users/HP/OneDrive/Desktop/DSA CSE25045/compound interest.py", line 7, in power
    return p * power(p, n - 1)
  [Previous line repeated 1023 more times]
RecursionError: maximum recursion depth exceeded
```

EXPLANATION:

- The program accepts the growth factor **p** and the number of years **n** as input and calls the recursive function power(p, n).

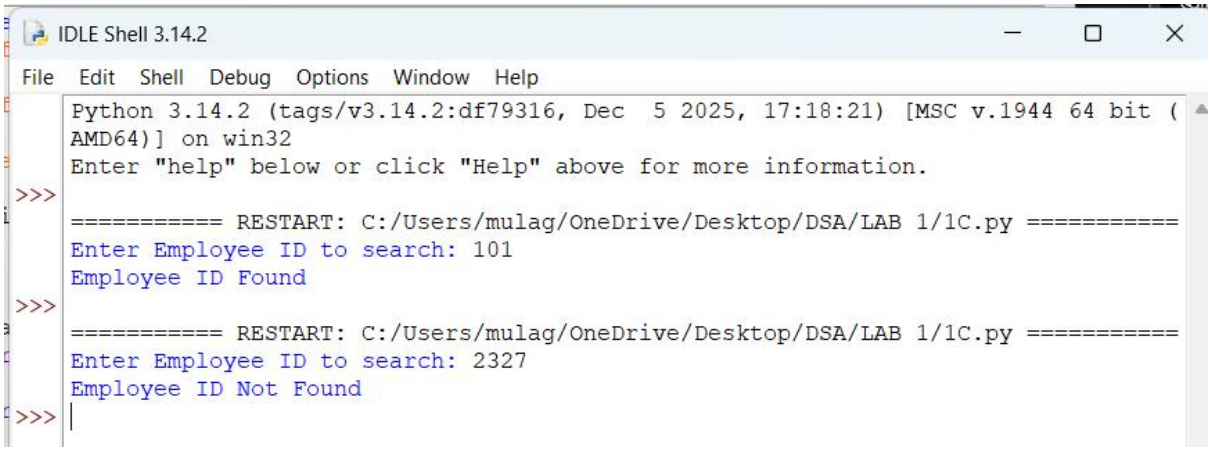
- The function checks the **base case** if n == 0; if true, it returns **1**, since any number raised to the power 0 is 1.
- If n > 0, the function recursively computes the result by returning p * power(p, n - 1) until the base case is reached.
- If n < 0, the recursive calls continue indefinitely because the base case is never satisfied, resulting in a **Recursion Error**.
- Finally, for valid non-negative inputs, the program displays the value of $\mathbf{p^n}$, representing the growth factor raised to the given number of years.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Invalid Input Error** I entered a floating-point value instead of an integer for the number of years.	It was rectified by entering an integer value.
2.	**Large Input Error** I entered a very large value for the number of years.	It was rectified by entering a smaller input value within Python's recursion limit.

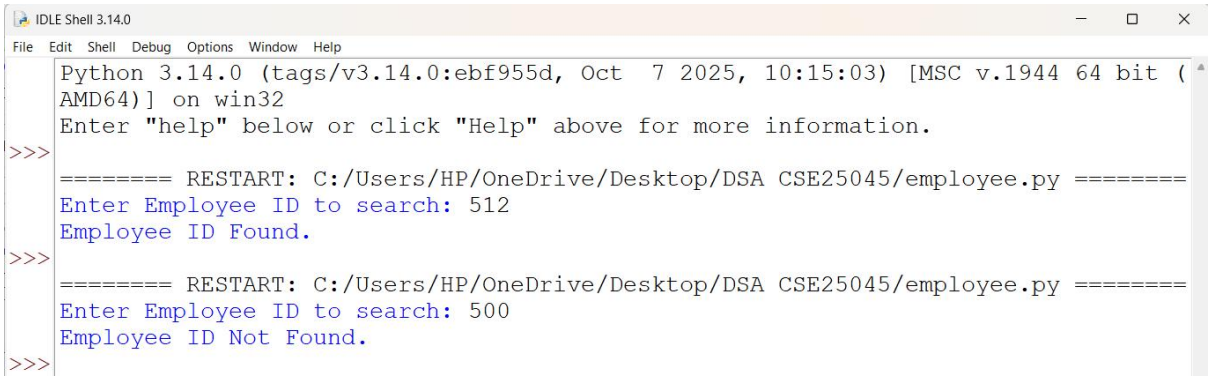
1c. AIM: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:



OUTPUT:

Positive Case and Negative Case;



EXPLANATION:

- The program accepts the **Employee ID** as input and invokes the recursive function search_employee(emp_list, target, index) with the initial index set to **0**.
- The function compares emp_list[index] with the target Employee ID. If they are equal, it returns **True**, terminating the recursion.
- If no match is found, the function recursively calls itself with index + 1 to search the next element in the list.
- The **base case** is reached when index == len(emp_list), indicating that all elements have been searched. The function then returns **False**.
- Based on the returned Boolean value, the main program displays either **"Employee ID Found ."** or **"Employee ID Not Found."**.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**List Initialization Error** I have not entered any Employee IDs in the list.	It was rectified by adding the required Employee IDs to the list before performing the search.
2.	**Index Error** I tried to access an index beyond the size of the list.	It was rectified by checking the base condition before accessing the list element.

1d. AIM: In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
1D.py - C:/Users/mulag/OneDrive/Desktop/DSA/LAB 1/1D.py (3.14.2)
File  Edit  Format  Run  Options  Window  Help
def factorial(n):
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)
n = int(input("Enter a number: "))

if n < 0:
    print("Invalid input! Factorial is not defined for negative numbers.")
else:
    print("Factorial =", factorial(n))
```

OUTPUT:

Positive Case;

```
IDLE Shell 3.14.0
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/factorial.py =======
Enter the Number:7
Possible ways to arrange 7 different parcels is 5040
```

Negative Case;

```
======================== RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/factorial.py ========================
Enter the Number:-24
Traceback (most recent call last):
  File "C:/Users/HP/OneDrive/Desktop/DSA CSE25045/factorial.py", line 7, in <module>
    print(f"Possible ways to arrange {n} different parcels is", factorial(n))
  File "C:/Users/HP/OneDrive/Desktop/DSA CSE25045/factorial.py", line 6, in factorial
    return n * factorial(n - 1)
  File "C:/Users/HP/OneDrive/Desktop/DSA CSE25045/factorial.py", line 6, in factorial
    return n * factorial(n - 1)
  File "C:/Users/HP/OneDrive/Desktop/DSA CSE25045/factorial.py", line 6, in factorial
    return n * factorial(n - 1)
  [Previous line repeated 1023 more times]
RecursionError: maximum recursion depth exceeded
>>>
```

EXPLANATION:

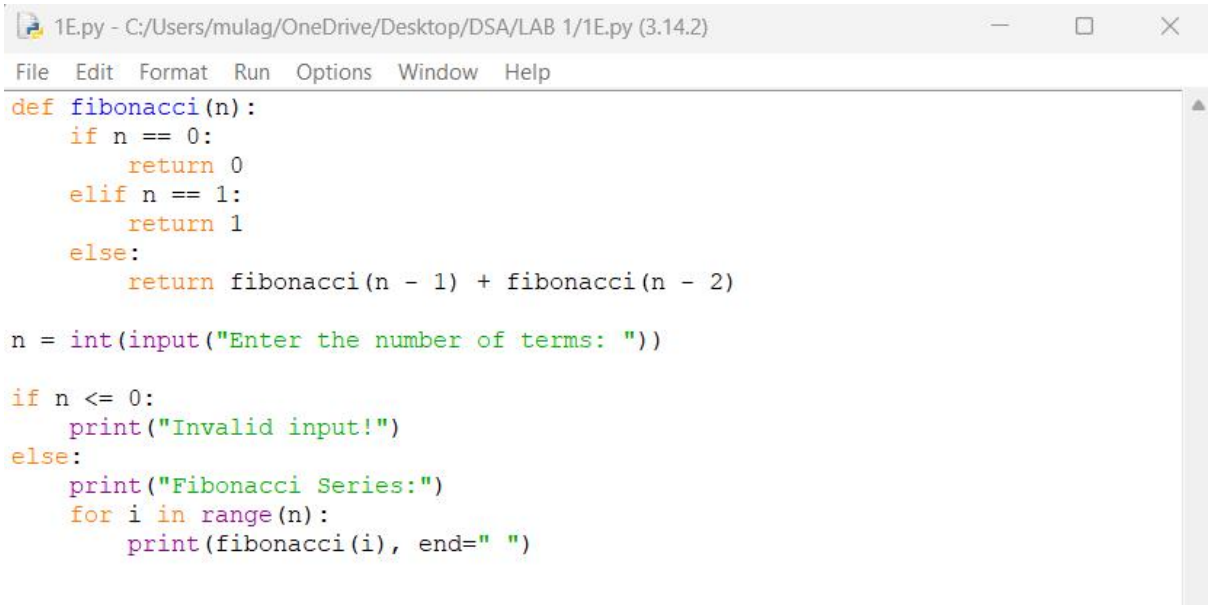
- The program accepts an integer **n** as input and calls the recursive function factorial(n).
- The function checks the **base case** if n == 0; if true, it returns **1**, since 0! = 1.
- If n > 0, the function recursively calls factorial(n - 1) and returns n * factorial(n - 1) to compute the factorial.
- The recursion continues until the base case is reached, after which the recursive calls return the final factorial value.
- Finally, the program displays the total number of possible arrangements (n!) for the given number of parcels.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Negative Input Error** I entered a negative integer as input.	It was rectified by entering a non-negative integer ($n \geq 0$).
2.	**Name Error** I called the function with an incorrect function name.	It was rectified by using the correct function name factorial().

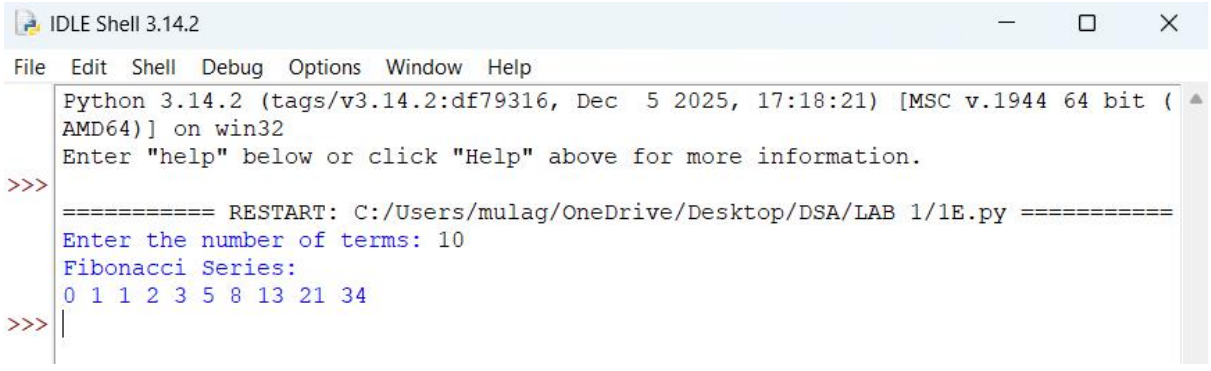
1e. AIM: Write a recursive python program to print the first n terms of the Fibonacci Series.

PROGRAM:



OUTPUT:

Positive Case;



Negative Case;



EXPLANATION:

- The program accepts an integer **n** as input and calls the recursive function fib(n).
- The function checks the **base case** if n <= 1; if true, it returns n, representing the first two Fibonacci numbers (0 and 1).
- If n > 1, the function recursively computes the Fibonacci number by returning fib(n - 1) + fib(n - 2).
- The recursion continues until the base case is reached, after which the recursive calls return and calculate the required Fibonacci number.
- Finally, the program displays the Fibonacci number corresponding to the given position n.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Negative Input Error** I entered a negative integer as input.	It was rectified by entering a non-negative integer ($n \geq 0$).
2.	**Large Input Error** I entered a very large number.	It was rectified by entering a smaller input value.

WEEK – 2

2a**. AIM:** Write a python program to perform linear search on the given input.

PROGRAM:

2A.py - C:/Users/mulag/OneDrive/Desktop/DSA/LAB 2/2A.py (3.14.2)

File Edit Format Run Options Window Help

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

n=int(input("Enter the no of elements: "))
arr=[]
print("enter the elements:")
for i in range(n):
    arr.append(int(input()))
key = int(input("Enter element to search: "))

result = linear_search(arr, key)

if result != -1:
    print("Element found at position", result )
else:
    print("Element not found")
```

OUTPUT:

Positive Case;

File Edit Shell Debug Options Window Help

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 2/2A.py ==========
Enter the no of elements: 6
enter the elements:
3
5
9
8
9
1
Enter element to search: 1
Element found at position 5
>>>
```

Negative Case;

```
>>>
==== RESTART: C:\Users\HP\OneDrive\Desktop\DSA CSE25045\Linear Search (1).py ===
Enter the number of elements:5
Enter elements in order:
15
20
25
30
45
Enter the element to search:55
Element not found
```

EXPLANATION:

- The program implements **Linear Search** to find a specific element in an array.
- It first asks the user to enter the **number of elements** and stores them in a list.
- The linear_search() function checks each element of the array **one by one**.
- If the result is not -1, it displays **"Element found at index"**.
- If the result is -1, it displays **"Element not found"**.
- Thus, the program demonstrates both **successful (positive)** and **unsuccessful (negative)** linear search cases.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Search Element Not Found:** The entered element was not present in the array.	**Search Element Not Found:** The entered element was not present in the array.
2.	**Wrong Search Result:** The program was given an incorrect search element.	Re-entered the correct element to be searched.

2b**. AIM:** Write a python program to perform binary search on the given input.

PROGRAM:

2b.py - C:/Users/mulag/OneDrive/Desktop/DSA/LAB 2/2b.py (3.14.2)

File Edit Format Run Options Window Help

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1
n = int(input("Enter number of elements: "))
arr = []
print("Enter", n, "elements:")
for i in range(n):
    arr.append(int(input()))

key = int(input("Enter element to search: "))

result = binary_search(arr, key)

if result != -1:
    print("Element found at position", result + 1)
else:
    print("Element not found")
```

OUTPUT:

Positive Case;

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
========== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 2/2b.py ==========
Enter number of elements: 5
Enter 5 elements:
10
20
30
40
50
Enter element to search: 40
Element found at position 4
>>
========== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/LAB 2/2b.py ==========
Enter number of elements: 5
Enter 5 elements:
50
10
30
20
40
Enter element to search: 40
Element found at position 5
>>
```

Negative Case;

```
>>>
==== RESTART: C:\Users\HP\OneDrive\Desktop\DSA CSE25045\Binary Search (1).py ===
Enter the number of elements:5
Enter elements in sorted order:
46
89
23
56
67
Enter the element to search:12
Element not found
```

EXPLANATION:

- The program defines a binary_search() function to search for a **key** in a sorted arr.
- It initializes low as 0 and high as len(arr) - 1.
- The while low <= high loop calculates the mid index using (low + high) // 2.
- If arr[mid] == key, the **mid index is returned**.
- If arr[mid] < key, low is updated to mid + 1; otherwise, high is updated to mid - 1.
- If the key is not found, the function returns **-1** and the program displays **"Element not found"**.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Unsorted Array:** Binary search gave an incorrect result when the elements were not in sorted order.	Entered the elements in **ascending order** before performing the search.
2.	**Incorrect Search Range:** The low and high values were not updated correctly after comparing with arr[mid].	Updated low = mid + 1 for the right half and high = mid - 1 for the left half.

2c**. AIM:** Verify by taking sorted array and unsorted array as an input.

PROGRAM:

```
*Binary Unsorted (1).py - C:\Users\HP\OneDrive\Desktop\DSA CSE25045\Binary Unsorted (1).py (3.14.0)*
File  Edit  Format  Run  Options  Window  Help
def binary_search(arr,key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
n = int(input("Enter the number of elements:"))
arr = []
print("Enter elements in sorted order:")
for i in range(n):
    arr.append(int(input()))
key = int(input("Enter the element to search:"))

if arr == sorted(arr):
    print("The input list is already sorted.")
else:
    print("The input list is not sorted.")
    print("The list is sorting...")
    arr.sort()
print(f"Sorted list: {arr}")

result = binary_search(arr,key)
if result != -1:
  print(f"Element found at index {result}")
else:
    print("Element not found")
```

OUTPUT:

Sorted Array;

```
IDLE Shell 3.14.0
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:\Users\HP\OneDrive\Desktop\DSA CSE25045\Binary Unsorted (1).py ==
Enter the number of elements:5
Enter elements in sorted order:
10
20
30
40
50
Enter the element to search:10
The input list is already sorted.
Sorted list: [10, 20, 30, 40, 50]
Element found at index 0
```

Unsorted Array;

```
>>>
=== RESTART: C:\Users\HP\OneDrive\Desktop\DSA CSE25045\Binary Unsorted (1).py ==
Enter the number of elements:5
Enter elements in sorted order:
40
10
50
30
20
Enter the element to search:10
The input list is not sorted.
The list is sorting...
Sorted list: [10, 20, 30, 40, 50]
Element found at index 0
```

EXPLANATION:

- The program defines a binary_search() function to search for a **key** in an array using Binary Search.
- It initializes low and high and calculates the mid index to compare the middle element with the key.
- It checks whether arr == sorted(arr) to determine if the input list is already **sorted**.
- If the list is not sorted, arr.sort() sorts the elements before performing the search.
- The binary_search() function searches the sorted arr and returns the **index** if the key is found, otherwise it returns **-1**.
- Finally, the program displays whether the **element is found or not found**.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Unsorted Array:** The input elements were entered in an unsorted order.	Used arr.sort() to arrange the elements in ascending order before binary search.
2.	**Incorrect Sorted Order:** The elements were not arranged in the required ascending order for binary search.	Checked the list using arr == sorted(arr) and sorted it when required.

WEEK – 3

3a**. AIM:** Write a python program to sort the elements using a bubble sort technique. Take the input elements from the user.

PROGRAM:

```
3.1.py - C:/Users/mulag/OneDrive/Desktop/DSA/lab 3/3.1.py (3.14.2)
File  Edit  Format  Run  Options  Window  Help
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
n = int(input("Enter the number of elements: "))
arr = []
for i in range(n):
    element = int(input(f"Enter element {i + 1}: "))
    arr.append(element)
bubble_sort(arr)
print("Sorted elements:", arr)
```

OUTPUT:

```
ile  Edit  Shell  Debug  Options  Window  Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64 bit
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>
========== RESTART: C:/Users/mulag/OneDrive/Desktop/DSA/lab 3/3.1.py =========
Enter the number of elements: 5
Enter element 1: 40
Enter element 2: 29
Enter element 3: 44
Enter element 4: 35
Enter element 5: 23
Sorted elements: [23, 29, 35, 40, 44]
>>
```

EXPLANATION:

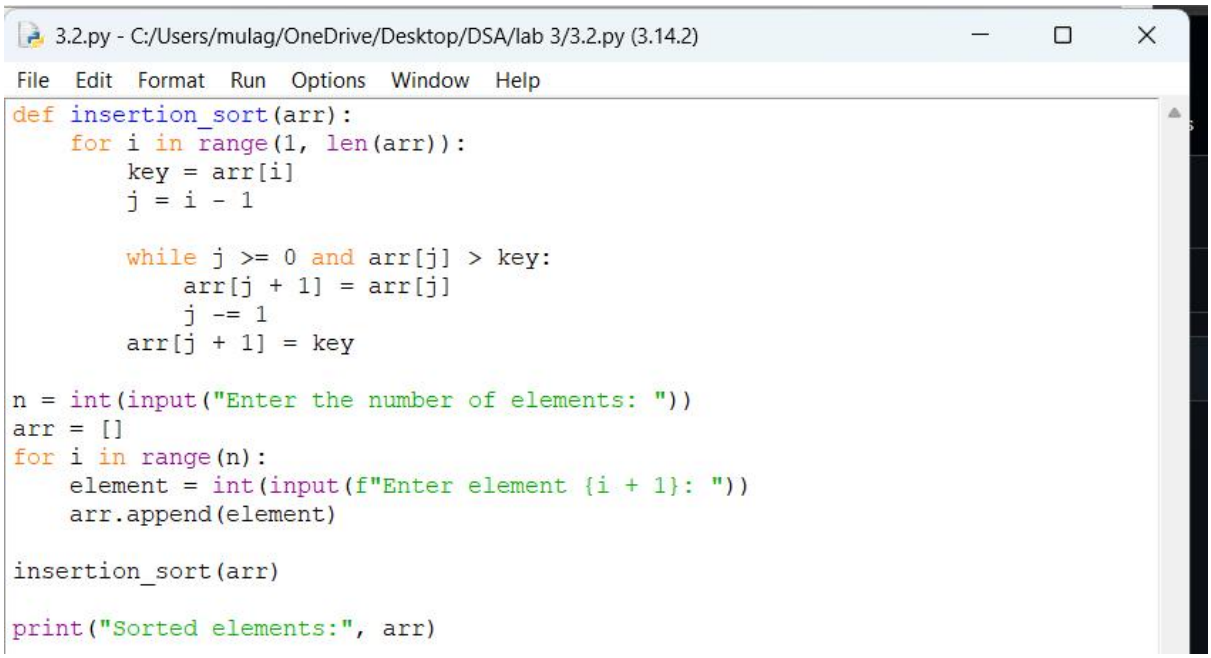
- n = int(input(...)) – Gets the number of elements.
- arr = [] – Creates an empty list to store elements.
- for i in range(n) – Takes n elements as input.
- arr.append(...) – Adds each element to arr.
- for i in range(n - 1) – Performs Bubble Sort passes.
- for j in range(n - i - 1) – Compares adjacent elements.
- if arr[j] < arr[j + 1] – Checks if the current element is smaller.
- arr[j], arr[j + 1] = ... – Swaps the elements to sort in descending order.
- print(arr) – Displays the sorted list.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Sorting Order Error** – Using > instead of < in the if condition changes the order to ascending.	Use if arr[j] < arr[j + 1]: for descending order.

3b**. AIM:** Write a python program to sort the elements using a insertion sort technique. Take the input elements from the user.

PROGRAM:



OUTPUT:

```
===== RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/insertion sort.py =====
Enter the number of elements: 4
Enter the elements:
67
80
34
45
Elements after Insertion Sort:
[34, 45, 67, 80]
```

EXPLANATION:

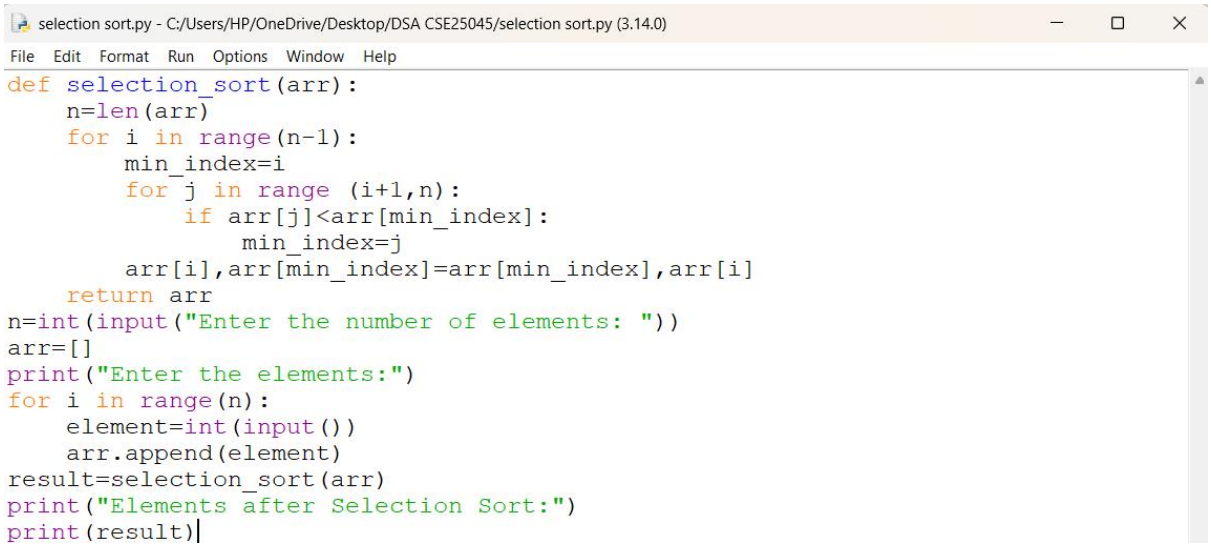
• n = len(arr) – Gets the list size.
• for i in range(1, n) – Selects elements from the second position.
• key = arr[i] – Stores the current element.
• j = i - 1 – Sets the previous index.
• while j >= 0 and arr[j] > key – Compares previous elements with key.
• arr[j + 1] = arr[j] – Shifts larger elements right.
• arr[j + 1] = key – Inserts key in the correct position.
• return arr – Returns the sorted list.
• result = insertion_sort(arr) – Calls the function.
• print(result) – Displays the sorted list.

ERROR TABLE:

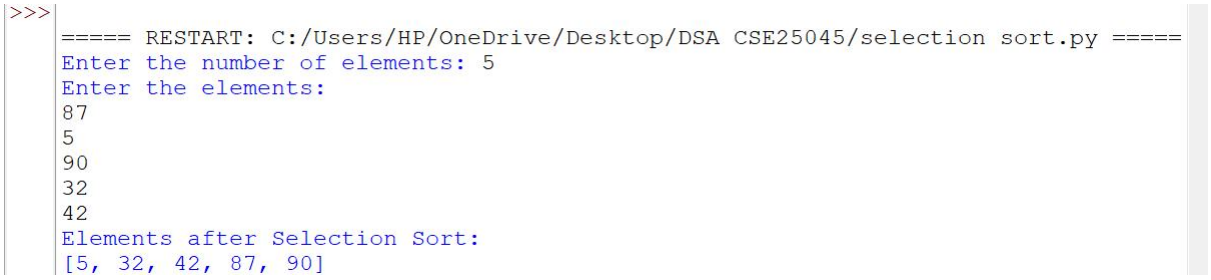
S.NO	Error Obtained	Rectification
1.	**Insertion Error** – Placing key at the wrong index.	Use arr[j + 1] = key after shifting elements.
2.	**Function Call Error** – Not calling insertion_sort(arr) correctly.	Pass the input list arr to the function.

3c**. AIM:** Write a python program to sort the elements using a selection sort technique. Take the input elements from the user.

PROGRAM:



OUTPUT:



EXPLANATION:

• def selection_sort(arr) – Defines the Selection Sort function.
• n = len(arr) – Gets the number of elements in arr.
• for i in range(n-1) – Performs sorting passes.
• min_index = i – Assumes the first element is the smallest.

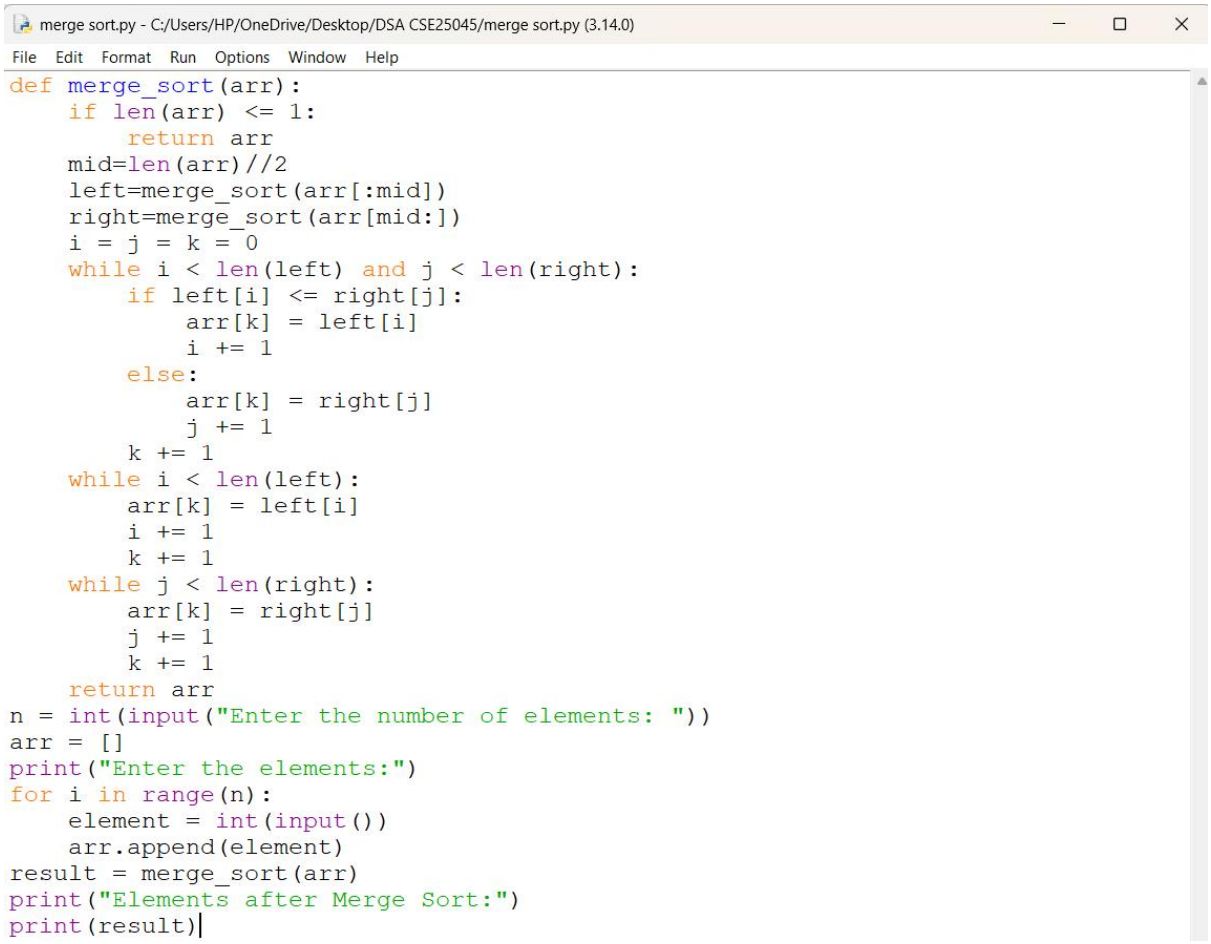
• for j in range(i+1,n) – Checks the remaining elements.
• if arr[j] < arr[min_index] – Finds the smallest element.
• arr[i], arr[min_index] = ... – Swaps the smallest element into position.
• return arr – Returns the sorted list.
• result = selection_sort(arr) – Calls the function.
• print(result) – Displays the sorted list in ascending order.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Minimum Index Error** – Incorrectly assigning min_index.	Use min_index = i.
2.	**Swap Error** – Incorrectly swapping the elements.	Use arr[i], arr[min_index] = arr[min_index], arr[i].

3d**. AIM:** Write a python program to sort the elements using a merge sort technique. Take the input elements from the user.

PROGRAM:



OUTPUT:

```
>>>
======= RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/merge sort.py =======
Enter the number of elements: 4
Enter the elements:
87
56
34
13
Elements after Merge Sort:
[13, 34, 56, 87]
```

EXPLANATION:

• def merge_sort(arr) – Defines the Merge Sort function.
• if len(arr) <= 1 – Stops when the list has one or no elements.
• mid = len(arr)//2 – Finds the middle of the list.
• left and right – Divide the list into two halves recursively.
• i = j = k = 0 – Initializes the indexes.
• while loops – Compare and merge the elements.
• if left[i] <= right[j] – Selects the smaller element.
• arr[k] – Places the selected element in the list.
• return arr – Returns the sorted list.
• print(result) – Displays the sorted list.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Recursion Error** – Incorrectly dividing the list.	Use arr[:mid] and arr[mid:].
2.	**Index Error** – Incorrectly updating i, j, or k.	Increment the appropriate index after each insertion.

3e**. AIM:** Write a python program to sort the elements using a quick sort technique. Take the input elements from the user.

PROGRAM:

quick sort.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/quick sort.py (3.14.0)

File Edit Format Run Options Window Help

```
def quicksort(a,low,high):
    if low < high:
        i = low
        j = high
        pivot = low
        while i < j:
            while i < len(a) and a[i]<=a[pivot]:
                i +=1
            while a[j] > a[pivot]:
                j = j -1
            if i < j:
                a[i],a[j] = a[j],a[i]
        a[j],a[pivot] = a[pivot],a[j]
        quicksort(a,low,j-1)
        quicksort(a,j+1,high)
a = list(map(int,input("Enter numbers to sort:").split(" ")))
n = len(a)
quicksort(a,0,n-1)
print("Sorted array:")
print(a)
```

OUTPUT:

```
>>>
======= RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/quick sort.py =======
Enter numbers to sort:63 33 99 26 11
Sorted array:
[11, 26, 33, 63, 99]
```

EXPLANATION:

- def quicksort(a, low, high) – Defines the Quick Sort function.
- if low < high – Checks the sorting range.
- i and j – Set left and right positions.
- pivot = low – Selects the first element as pivot.
- while i < j – Performs partitioning.
- Inner while loops – Find elements for swapping.
- a[i], a[j] = ... – Swaps elements.
- a[j], a[pivot] = ... – Places the pivot correctly.
- quicksort() calls – Sort the two partitions.
- input().split() – Takes space-separated numbers.
- print(a) – Displays the sorted array.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**Pivot Error** – Incorrectly selecting the pivot index.	Use pivot = low.
2.	**Swap Error** – Incorrect swapping of a[i] and a[j].	Use a[i], a[j] = a[j], a[i].

WEEK – 4

4a**. AIM:** Write a python program to implement a SLL which could perform the below operations by taking an input from the user.

1) create a LL

2) Insert at beginning

3) Insert at the end

4) Insert at specific position

5) delete by value

6) delete first node

7) delete last node

8) Count no. of nodes

9) display / traverse

10) exit

PROGRAM:

single linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/single linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    def create(self, n):
        self.head = None

        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            self.insert_end(data)

        print("Linked List created successfully")

    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new

    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
        else:
            temp = self.head

            while temp.next:
                temp = temp.next

            temp.next = new

    def insert_position(self, data, position):
        new = Node(data)

        if position < 1:
            print("Invalid position")
```


single linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/single linked list.py (3.14.0)
File Edit Format Run Options Window Help
return
if position == 1:
new.next = self.head
self.head = new
print("Node inserted successfully")
return
temp = self.head
for i in range(position - 2):
if temp is None:
print("Invalid position")
return
temp = temp.next
if temp is None:
print("Invalid position")
else:
new.next = temp.next
temp.next = new
print("Node inserted successfully")
def delete_value(self, value):
if self.head is None:
print("No Data to delete")
return
if self.head.data == value:
self.head = self.head.next
print("Value deleted")
return
temp = self.head
while temp.next and temp.next.data != value:
temp = temp.next
if temp.next is None:
print("Value not present")
else:
temp.next = temp.next.next
print("Value deleted")
Ln: 21 Col: 24


single linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/single linked list.py (3.14.0)
File Edit Format Run Options Window Help
def delete_first(self):
if self.head is None:
print("No Data to delete")
else:
print("Deleted Value =", self.head.data)
self.head = self.head.next
def delete_last(self):
if self.head is None:
print("No Data to delete")
elif self.head.next is None:
print("Deleted Value =", self.head.data)
self.head = None
else:
temp = self.head
while temp.next.next:
temp = temp.next
print("Deleted Value =", temp.next.data)
temp.next = None
def count(self):
if self.head is None:
print("No Linked List")
else:
c = 0
temp = self.head
while temp:
c += 1
temp = temp.next
print("Number of nodes =", c)
def display(self):
if self.head is None:
print("No Linked List")
else:
temp = self.head
Ln: 84 Col: 34

single linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/single linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
            while temp:
                print(temp.data, end=" -> ")
                temp = temp.next

            print("None")

ll = LinkedList()

while True:
    print("\n--- Singly Linked List Operations ---")
    print("1. Create a Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Specific Position")
    print("5. Delete a Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display Nodes")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        n = int(input("Enter number of nodes: "))
        ll.create(n)

    elif choice == 2:
        data = int(input("Enter data: "))
        ll.insert_begin(data)

    elif choice == 3:
        data = int(input("Enter data: "))
        ll.insert_end(data)

    elif choice == 4:
        data = int(input("Enter data: "))
        position = int(input("Enter position: "))
        ll.insert_position(data, position)


    elif choice == 5:
        value = int(input("Enter value to delete: "))
        ll.delete_value(value)

    elif choice == 6:
        ll.delete_first()

    elif choice == 7:
        ll.delete_last()

    elif choice == 8:
        ll.count()

    elif choice == 9:
        ll.display()

    elif choice == 10:
        print("Program ended")
        break

    else:
        print("Invalid choice")
```

OUTPUT:



EXPLANATION:

1. class Node

- Defines a **node** of the Singly Linked List.
- self.data stores the value of the node.
- self.next stores the reference to the next node.

2. class LinkedList

- Defines the **Linked List** and its operations.
- self.head points to the **first node** of the list.

3. create()

- Creates n nodes using a for loop.
- Takes data from the user and calls insert_end() to add each node.

4. insert_begin()

- Creates a new node using Node(data).
- new.next = self.head links it to the current first node.
- Updates self.head to make the new node the first node.

5. insert_end()

- Creates a new node.
- Uses while temp.next to **traverse** to the last node.
- Connects the new node using temp.next = new.

6. insert_position()

- Inserts a node at a specific position.
- Traverses to the required location using a loop.
- Updates the next references to connect the new node correctly.

7. delete_value()

- Searches for a node containing the given value.
- Uses a while loop for traversal.
- Deletes the node by changing the appropriate next reference.

8. delete_first()

- Deletes the first node.
- Updates the head using:
  self.head = self.head.next.

9. delete_last()

- Traverses to the **second-last node**.
- Sets temp.next = None to remove the last node.
- Also handles the special case of a single-node list.

10. count()

- Traverses the linked list using temp.
- Increments counter c for every node.

- Displays the total number of nodes.

11. display()

- Traverses the list from head until None.
- Prints the data of each node in linked list format.

12. Main Program

- ll = LinkedList() creates an **object** of the LinkedList class.
- while True creates an infinite loop to repeatedly display the menu.
- if-elif statements perform operations based on the user's choice.
- break terminates the loop and ends the program when **Exit** is selected.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Invalid insertion position.	Added position validation using if position < 1 and if temp is None.
2.	Value to be deleted may not exist.	Checked using if temp.next is None and displayed "Value not present".
3.	Program may terminate unexpectedly during normal operations.	Used proper conditional statements and return statements to control program flow.

4b**. AIM:** Write a python program to implement a DLL which could perform the below operations by taking an input from the user.

1) create a DLL

2) Insert at beginning

3) Insert at the end

4) Insert at specific position

5) deletion at beginning

6) deletion at the end

7) deletion at the specific position

8) Count no. of nodes

9) display / traverse

10) exit

PROGRAM:

double linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/double linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head

                while temp.next is not None:
                    temp = temp.next

                temp.next = new_node
                new_node.prev = temp
        print("Linked List created successfully.")

    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node
        print("Node inserted at beginning.")
```

double linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/double linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
    def insert_end(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head

            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

        print("Node inserted at end.")

    def insert_position(self):
        position = int(input("Enter position: "))
        data = int(input("Enter data: "))

        if position <= 0:
            print("Invalid position.")
            return

        if position == 1:
            new_node = Node(data)

            if self.head is not None:
                new_node.next = self.head
                self.head.prev = new_node

            self.head = new_node
            print("Node inserted.")
            return

        temp = self.head

        for i in range(1, position - 1):
            if temp is None:
                print("Invalid position.")
                return
```

double linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/double linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
            temp = temp.next

        if temp is None:
            print("Invalid position.")
            return

        new_node = Node(data)

        new_node.next = temp.next
        new_node.prev = temp

        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node

        print("Node inserted.")

    def delete_beginning(self):
        if self.head is None:
            print("Linked List is empty.")
            return

        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("Node deleted from beginning.")

    def delete_end(self):
        if self.head is None:
            print("Linked List is empty.")
            return

        temp = self.head

        if temp.next is None:
            self.head = None
            print("Node deleted from end.")
            return
```

double linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/double linked list.py (3.14.0)

File Edit Format Run Options Window Help

```
        while temp.next is not None:
            temp = temp.next

        temp.prev.next = None

        print("Node deleted from end.")

    def delete_position(self):
        position = int(input("Enter position: "))

        if self.head is None:
            print("Linked List is empty.")
            return

        if position <= 0:
            print("Invalid position.")
            return

        if position == 1:
            self.head = self.head.next

            if self.head is not None:
                self.head.prev = None

            print("Node deleted.")
            return

        temp = self.head

        for i in range(1, position):
            if temp is None:
                print("Invalid position.")
                return
            temp = temp.next

        if temp is None:
            print("Invalid position.")
            return

        if temp.next is not None:
            temp.next.prev = temp.prev
```

```
*double linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/double linked list.py (3.14.0)*
File  Edit  Format  Run  Options  Window  Help
        if temp.prev is not None:
            temp.prev.next = temp.next
            |
        print("Node deleted.")

    def count(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next
        print("Number of nodes:", count)

    def display(self):
        if self.head is None:
            print("Linked List is empty.")
            return
        temp = self.head
        print("Doubly Linked List:")
        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")

dll = DoublyLinkedList()

while True:
    print("\n----- DOUBLY LINKED LIST -----")
    print("1. Create a LL")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific position")
    print("5. Deletion at beginning")
    print("6. Deletion at end")
    print("7. Deletion at specific position")
    print("8. Count no. of nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        dll.create()

    elif choice == 2:
        dll.insert_beginning()

    elif choice == 3:
        dll.insert_end()

    elif choice == 4:
        dll.insert_position()

    elif choice == 5:
        dll.delete_beginning()

    elif choice == 6:
        dll.delete_end()

    elif choice == 7:
        dll.delete_position()

    elif choice == 8:
        dll.count()

    elif choice == 9:
        dll.display()

    elif choice == 10:
        print("Program exited.")
        break

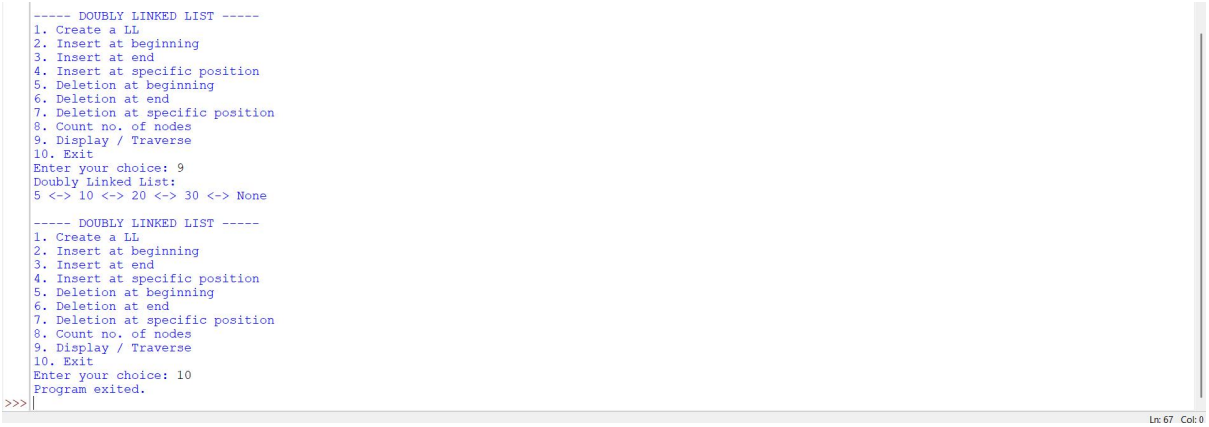
    else:
        print("Invalid choice. Please try again.")
                                                                    Ln: 170  Col: 12
```

OUTPUT:

```
IDLE Shell 3.14.0
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
=== RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/double linked list.py ===

----- DOUBLY LINKED LIST -----
1. Create a LL
2. Insert at beginning
3. Insert at end
4. Insert at specific position
5. Deletion at beginning
6. Deletion at end
7. Deletion at specific position
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter data: 10
Enter data: 20
Enter data: 30
Linked List created successfully.

----- DOUBLY LINKED LIST -----
1. Create a LL
2. Insert at beginning
3. Insert at end
4. Insert at specific position
5. Deletion at beginning
6. Deletion at end
7. Deletion at specific position
8. Count no. of nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter data: 5
Node inserted at beginning.
```



EXPLANATION:

- The program first defines a **Node class** to create nodes and initializes data, prev, and next for each node.
- The **DoublyLinkedList class** initializes an empty list by setting head to None.
- In **create()**, the user enters the number of nodes and their data. Each new node is added to the end of the list and connected in both directions.
- In **insert_beginning()**, a new node is created and connected before the current first node. The head is then changed to the new node.
- In **insert_end()**, the program starts from head, traverses to the last node, and attaches the new node after it.
- In **insert_position()**, the program traverses to the required position and places the new node between two existing nodes by updating their connections.
- In **delete_beginning()**, the first node is removed by moving head to the next node and making its prev pointer None.
- In **delete_end()**, the program traverses to the last node and removes it by disconnecting it from its previous node.
- In **delete_position()**, the program reaches the node at the given position and connects its previous and next nodes directly, removing the selected node.
- In **count()**, the program starts from head, visits every node using next, and increments count until the end is reached.
- In **display()**, every node is visited from the first node to the last node and its data is printed in the form 10 <-> 20 <-> 30.
- The **main while True loop** repeatedly displays all available operations and takes the user's choice.
- The **if-elif statements** call the appropriate function according to the selected option.

- When the user enters **10**, the break statement stops the loop and terminates the program.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	AttributeError: 'NoneType' object has no attribute 'next' may occur when trying to access temp.next while the list is empty.	Check whether self.head or temp is None before accessing next.
2.	Incorrect linking may occur during insertion or deletion if only one of the prev or next pointers is updated.	Update both prev and next pointers whenever a node is inserted or deleted.

4c**. AIM:** Write a python program to implement a CLL which could perform the below operations by taking an input from the user.

1) create a CLL

2) Insert at beginning

3) Insert at the end

4) Insert at specific position

5) deletion at beginning

6) deletion at the end

7) deletion at specific positon

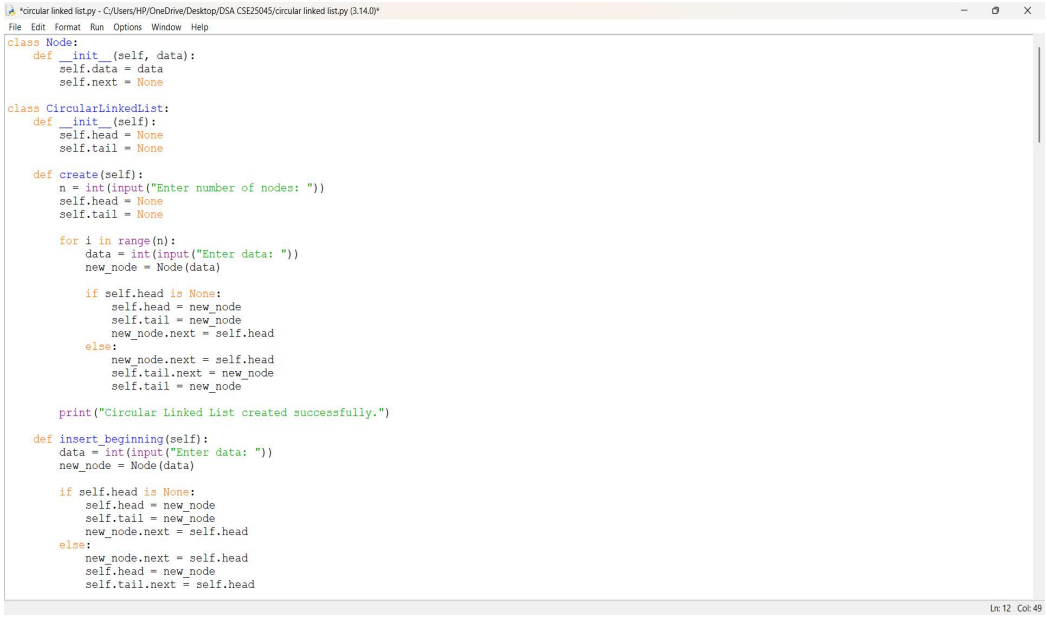
8) Count no. of nodes

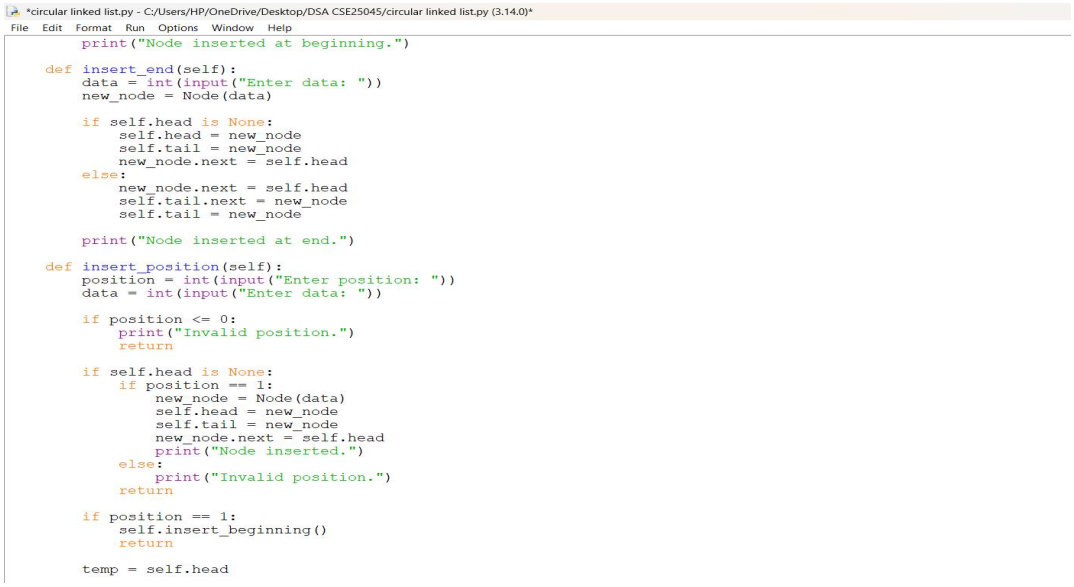
9) display / traverse

10) display head and tail

11) exit

PROGRAM:


circular linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/circular linked list.py (3.14.0)
File Edit Format Run Options Window Help
class Node:
def __init__(self, data):
self.data = data
self.next = None
class CircularLinkedList:
def __init__(self):
self.head = None
self.tail = None
def create(self):
n = int(input("Enter number of nodes: "))
self.head = None
self.tail = None
for i in range(n):
data = int(input("Enter data: "))
new_node = Node(data)
if self.head is None:
self.head = new_node
self.tail = new_node
new_node.next = self.head
else:
new_node.next = self.head
self.tail.next = new_node
self.tail = new_node
print("Circular Linked List created successfully.")
def insert_beginning(self):
data = int(input("Enter data: "))
new_node = Node(data)
if self.head is None:
self.head = new_node
self.tail = new_node
new_node.next = self.head
else:
new_node.next = self.head
self.head = new_node
self.tail.next = self.head
Ln: 12 Col: 49


circular linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/circular linked list.py (3.14.0)
File Edit Format Run Options Window Help
print("Node inserted at beginning.")
def insert_end(self):
data = int(input("Enter data: "))
new_node = Node(data)
if self.head is None:
self.head = new_node
self.tail = new_node
new_node.next = self.head
else:
new_node.next = self.head
self.tail.next = new_node
self.tail = new_node
print("Node inserted at end.")
def insert_position(self):
position = int(input("Enter position: "))
data = int(input("Enter data: "))
if position <= 0:
print("Invalid position.")
return
if self.head is None:
if position == 1:
new_node = Node(data)
self.head = new_node
self.tail = new_node
new_node.next = self.head
print("Node inserted.")
else:
print("Invalid position.")
return
if position == 1:
self.insert_beginning()
return
temp = self.head


circular linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/circular linked list.py (3.14.0)
File Edit Format Run Options Window Help
for i in range(1, position - 1):
temp = temp.next
if temp == self.head:
print("Invalid position.")
return
new_node = Node(data)
new_node.next = temp.next
temp.next = new_node
if temp == self.tail:
self.tail = new_node
print("Node inserted.")
def delete_beginning(self):
if self.head is None:
print("Circular Linked List is empty.")
return
if self.head == self.tail:
print("Deleted Value =", self.head.data)
self.head = None
self.tail = None
else:
print("Deleted Value =", self.head.data)
self.head = self.head.next
self.tail.next = self.head
def delete_end(self):
if self.head is None:
print("Circular Linked List is empty.")
return
if self.head == self.tail:
print("Deleted Value =", self.tail.data)
self.head = None
self.tail = None
return
temp = self.head
Ln: 12 Col: 49

```
        while temp.next != self.tail:
            temp = temp.next

        print("Deleted Value =", self.tail.data)

        temp.next = self.head
        self.tail = temp

    def delete_position(self):
        position = int(input("Enter position: "))

        if self.head is None:
            print("Circular Linked List is empty.")
            return
        if position <= 0:
            print("Invalid position.")
            return

        if position == 1:
            self.delete_beginning()
            return

        temp = self.head

        for i in range(1, position - 1):
            temp = temp.next

            if temp == self.head:
                print("Invalid position.")
                return

        if temp.next == self.head:
            print("Invalid position.")
            return

        if temp.next == self.tail:
            print("Deleted Value =", self.tail.data)
            self.tail = temp
            self.tail.next = self.head
        else:
            print("Deleted Value =", temp.next.data)
            temp.next = temp.next.next
```

```
    def count(self):
        if self.head is None:
            print("No Linked List")
            return

        count = 0
        temp = self.head

        while True:
            count += 1
            temp = temp.next

            if temp == self.head:
                break

        print("Number of nodes:", count)

    def display(self):
        if self.head is None:
            print("Circular Linked List is empty.")
            return

        temp = self.head

        print("Circular Linked List:")

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

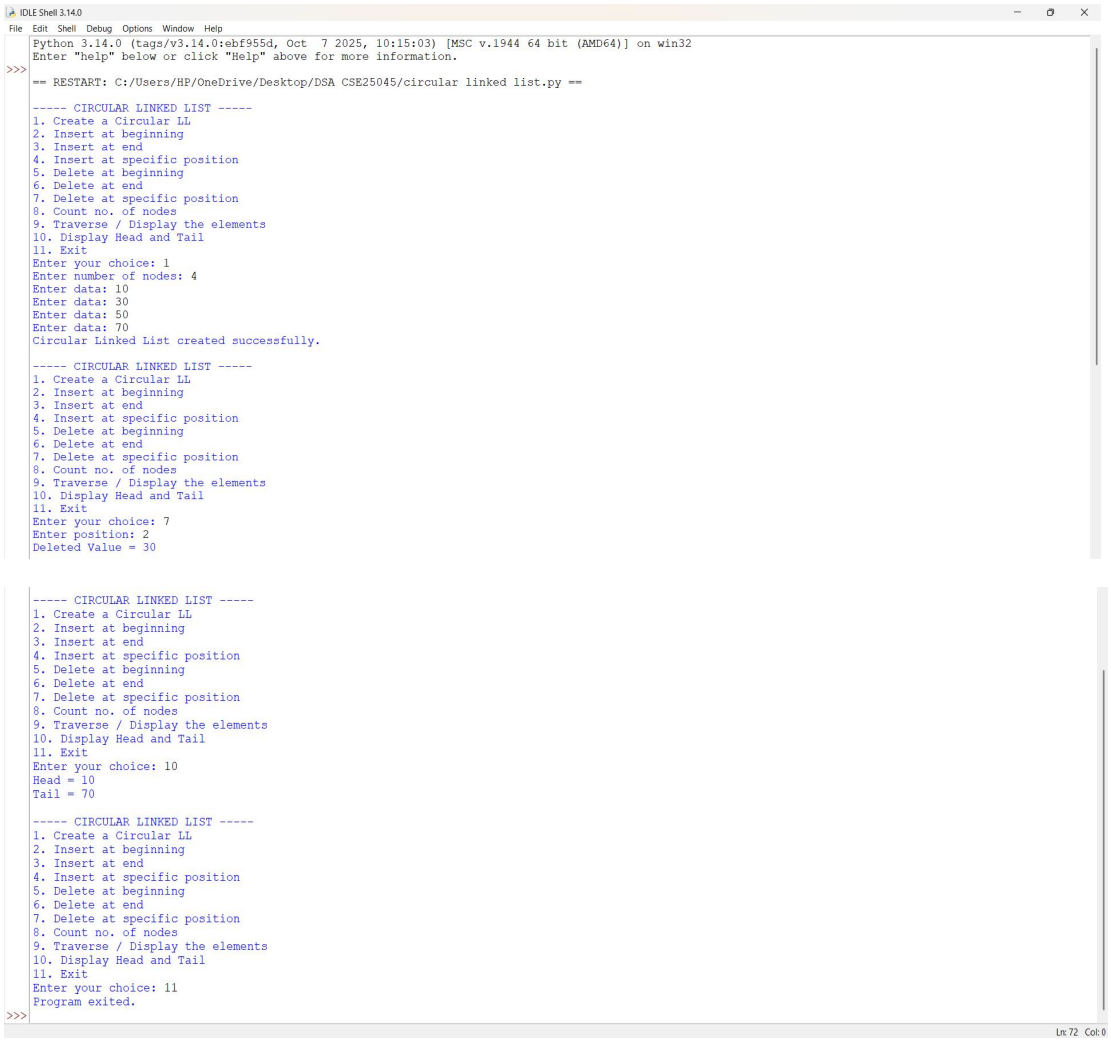
        print("(Head)")

    def display_head_tail(self):
        if self.head is None:
            print("Circular Linked List is empty.")
        else:
            print("Head =", self.head.data)
            print("Tail =", self.tail.data)
```

```
cll = CircularLinkedList()
while True:
    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create a Circular LL")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific position")
    print("5. Delete at beginning")
    print("6. Delete at end")
    print("7. Delete at specific position")
    print("8. Count no. of nodes")
    print("9. Traverse / Display the elements")
    print("10. Display Head and Tail")
    print("11. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        cll.create()
    elif choice == 2:
        cll.insert_beginning()
    elif choice == 3:
        cll.insert_end()
    elif choice == 4:
        cll.insert_position()
    elif choice == 5:
        cll.delete_beginning()
    elif choice == 6:
        cll.delete_end()
    elif choice == 7:
        cll.delete_position()
    elif choice == 8:
        cll.count()
    elif choice == 9:
        cll.display()
    elif choice == 10:
        cll.display_head_tail()
    elif choice == 11:
        print("Program exited.")
        break

    else:
        print("Invalid choice. Please try again.")
```

OUTPUT:



EXPLANATION:

- The program defines a **Node class** with data and next to store the value and link to the next node.
- The **CircularLinkedList class** maintains two references: head for the first node and tail for the last node.
- In **create()**, the user enters the number of nodes and their data. Each node is added to the list, and tail.next is always connected back to head.
- **insert_beginning()** creates a node before the current head and then updates tail.next so the circular connection is maintained.
- **insert_end()** adds a node after the current tail, makes it the new tail, and connects it back to head.
- **insert_position()** traverses the list up to the required position and inserts the new node between existing nodes. If inserted at the end, tail is updated.

- **delete_beginning()** removes the first node by moving head to the next node and reconnecting tail.next to the new head.
- **delete_end()** traverses to the node before tail, removes the last node, and makes that node the new tail.
- **delete_position()** finds the required node and bypasses it by changing the previous node's next. If the deleted node is the tail, tail is updated.
- **count()** starts from head and keeps traversing until it reaches head again, counting each node.
- **display()** prints every node starting from head and stops when the traversal reaches head again, avoiding an infinite loop.
- **display_head_tail()** directly prints the data stored in the head and tail nodes.
- The **while True loop** creates a menu-driven program where the user selects an operation using choice.
- The corresponding **if-elif statements** call the selected function.
- Selecting **11** executes break and terminates the program.
- The key feature of the code is that **tail.next = head**, which keeps the linked list circular.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	**infinite loop during traversal/display** because a circular linked list does not have a None at the end.	Use if temp == self.head: break to stop traversal when the pointer reaches the head again.
2.	**Incorrect circular connection** after insertion or deletion if tail.next is not updated.	Maintain self.tail.next = self.head after every insertion or deletion so that the last node always points back to the first node.

WEEK – 5

5**. AIM:** Write a python program to implement a stack using an Array, Linked list. Implement the stack operations such as push, pop, peak and also display the stack elements.

PROGRAM: Stack using an Array

stacks using arrays.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/stacks using arrays.py (3.14.0)

File Edit Format Run Options Window Help

```
class StackArray:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            for i in range(self.top, -1, -1):
                print(self.stack[i])


size = int(input("Enter size of stack: "))
s = StackArray(size)
```

```
while True:
    print("\n--- STACK USING ARRAY ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Exiting program...")
        break
    else:
        print("Invalid choice!. Try again.")
```

Ln: 55 Col: 21

OUTPUT:

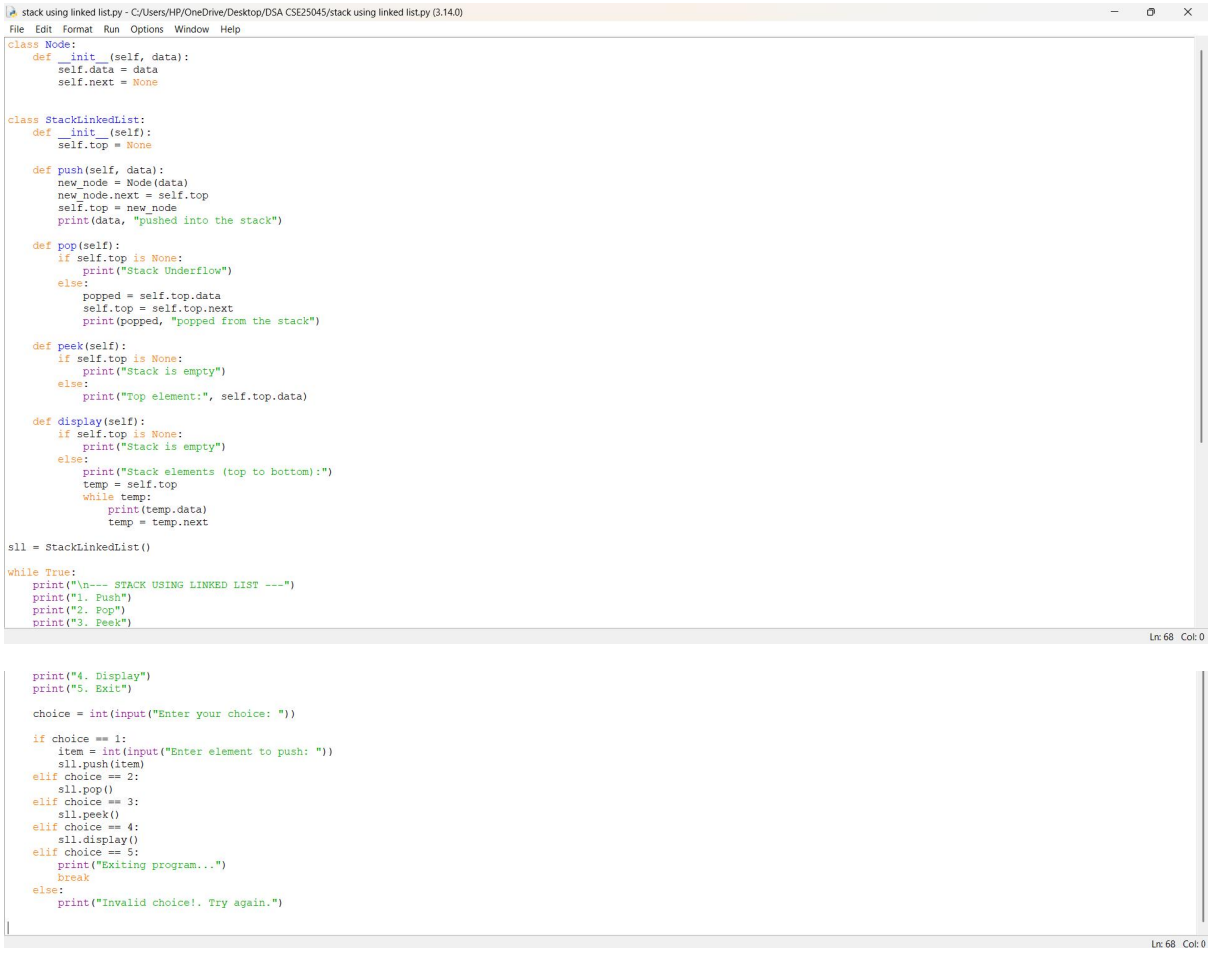


EXPLANATION:

- class StackArray: : Defines a class to implement the stack.
- __init__(size) : Creates the stack and sets its initial values.
- self.stack = [None] * size : Creates an array with empty positions.
- self.top = -1 : Sets top to -1 because the stack is initially empty.
- push(item) : Adds an element to the stack.
    - Checks if the stack is full.
    - Increases top by 1.
    - Stores the item at stack[top].
- pop() : Removes the top element.
    - Checks if the stack is empty.
    - Gets the element at stack[top].
    - Removes it by setting it to None.
    - Decreases top by 1.
- peek() : Shows the top element without removing it.
- display() : Uses a for loop to print elements from top to 0.
- size = int(input(...)) : Gets the stack size from the user.
- s = StackArray(size) : Creates a stack object.
- while True : Continuously displays the menu.

- choice : Takes the user's operation choice.
- LIFO principle : The last element pushed is the first element popped.

PROGRAM: Stack using an Linked List



```
stack using linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/stack using linked list.py (3.14.0)
File  Edit  Format  Run  Options  Window  Help
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class StackLinkedList:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(data, "pushed into the stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            popped = self.top.data
            self.top = self.top.next
            print(popped, "popped from the stack")

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            temp = self.top
            while temp:
                print(temp.data)
                temp = temp.next

sll = StackLinkedList()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
Ln: 68  Col: 0
```

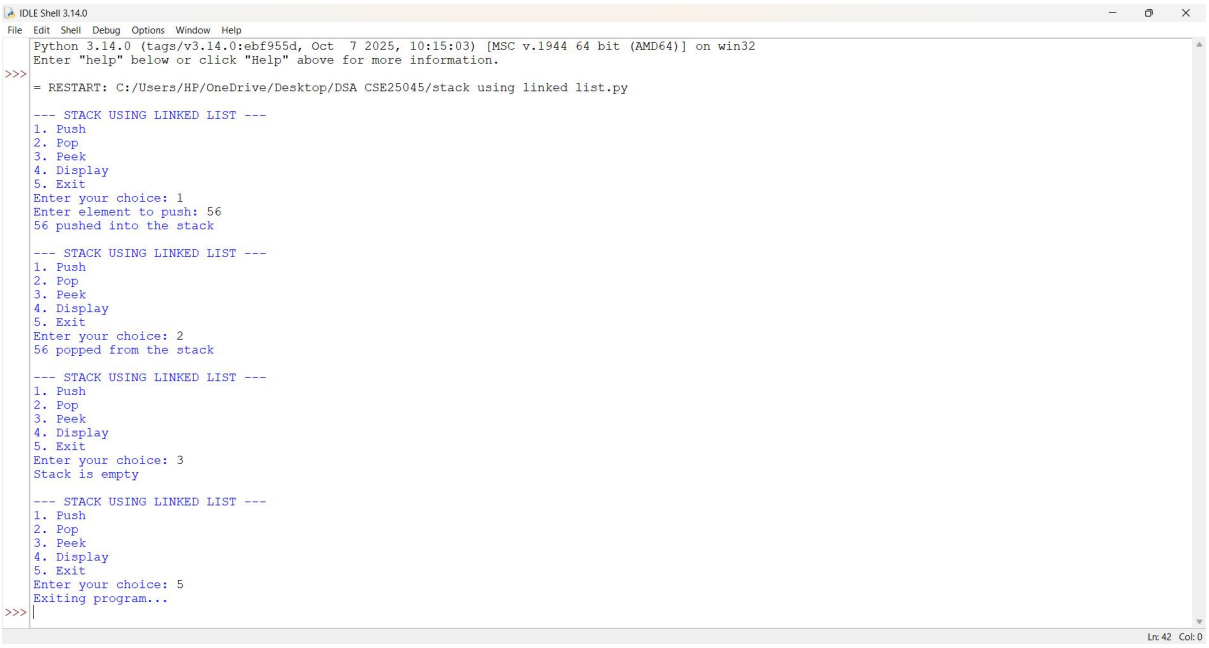
```
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to push: "))
        sll.push(item)
    elif choice == 2:
        sll.pop()
    elif choice == 3:
        sll.peek()
    elif choice == 4:
        sll.display()
    elif choice == 5:
        print("Exiting program...")
        break
    else:
        print("Invalid choice!. Try again.")

Ln: 68  Col: 0
```

OUTPUT:



EXPLANATION:

- Node(data) : Creates a node and stores the element in data.
- next = None : Initially, the node has no link.
- top = None : Initializes an empty stack.

Push:

- new_node = Node(data) : Creates a new node.
- new_node.next = self.top : Links it to the current top.
- self.top = new_node : Makes it the new top.
- **Process:** Create : Link : Update top.

Pop:

- Checks self.top is None for underflow.
- popped = self.top.data : Gets the top value.
- self.top = self.top.next : Moves top to the next node.
- **Process:** Get value : Move top : Remove old top.

Peek:

- self.top.data : Displays the top value without removing it.

Display:

- temp = self.top : Starts from top.
- while temp : Traverses the nodes.

- temp = temp.next : Moves to the next node.

Main Program:

- sll = StackLinkedList() : Creates the stack.
- choice : Selects Push, Pop, Peek, Display, or Exit.
- while True : Repeats the menu.
- break : Exits the program.
- **LIFO:** Last inserted : First removed.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	IndexError may occur if an element is accessed outside the array limit in the Array Stack.	Check whether the stack is full before push() and empty before pop() using top.
2.	AttributeError may occur in Linked List Stack if self.top.data is accessed when top is None.	Check if self.top is None before performing pop() or peek().

WEEK – 6

6a**. AIM:** Write a python program to implement queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach implement using array, linked list.

PROGRAM: Using an array

queue using arrays.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/queue using arrays.py (3.14.0)

File Edit Format Run Options Window Help

```
class QueueEx:
    def __init__(self, size=5):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
            self.queue[self.rear] = x
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            x = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1
            print(f"{x} deleted from the queue")

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
```

queue using arrays.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/queue using arrays.py (3.14.0)

File Edit Format Run Options Window Help

```
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])


size = int(input("Enter the size of Queue: "))
q = QueueEx(size)

while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

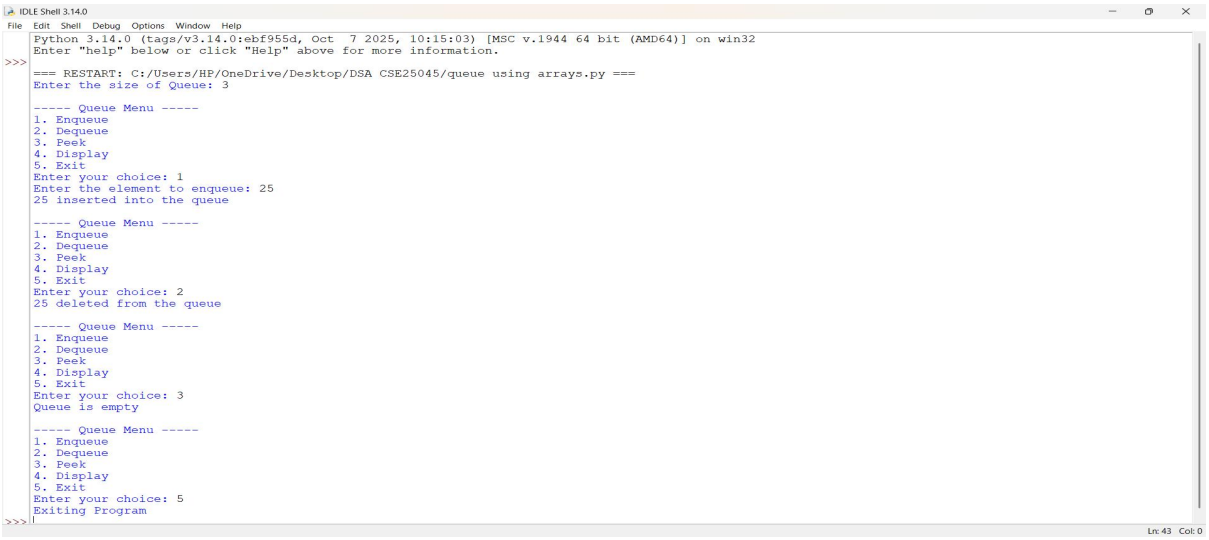
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
        print("Invalid choice")
```

Ln: 27 Col: 12

OUTPUT:



PROGRAM: Using linked list



```
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])


size = int(input("Enter the size of Queue: "))
q = QueueEx(size)

while True:
        print("\n----- Queue Menu -----")
        print("1. Enqueue")
        print("2. Dequeue")
        print("3. Peek")
        print("4. Display")
        print("5. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            item = int(input("Enter the element to enqueue: "))
            q.enqueue(item)

        elif choice == 2:
            q.dequeue()

        elif choice == 3:
            q.peek()

        elif choice == 4:
            q.display()

        elif choice == 5:
            print("Exiting Program")
            break

        else:
            print("Invalid Choice!")
```

OUTPUT:

```
===== RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/queue using ll.py =====
Enter the size of Queue: 3

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 86
86 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 56
56 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
86
56

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting Program
>>>
```

EXPLANATION:

- class QueueEx: Creates a **class** named QueueEx for implementing the queue.
- __init__(self, size=5): **Constructor** that initializes the queue with a default size of 5.
- self.size = size : Stores the user-given queue size.
- self.queue = [None] * size : Creates a list of the given size and initializes all positions with None.
- self.front = -1 : Initializes front to indicate that the queue is empty.
- self.rear = -1 : Initializes rear to indicate that no element is present.

enqueue()

- if self.rear == self.size - 1: Checks whether rear has reached the last position.
- self.front = 0 : Sets the front position when inserting the first element.
- self.rear += 1 : Increases rear to the next position.
- self.queue[self.rear] = x : Stores x at the rear position.
- print() : Displays the insertion message.

dequeue()

- if self.front == -1 or self.front > self.rear: Checks whether the queue is empty.
- x = self.queue[self.front] : Stores the front element in x.
- self.queue[self.front] = None : Removes the front element from the list.
- self.front += 1 : Moves front to the next position.
- if self.front > self.rear: : Checks whether all elements have been deleted.
- self.front = -1 and self.rear = -1 : Resets the queue to the empty state.

peek()

- if self.front == -1: Checks whether the queue is empty.
- self.queue[self.front] : Accesses and displays the front element without deleting it.

display()

- if self.front == -1: Checks whether the queue is empty.
- range(self.front, self.rear + 1) : Traverses the queue from front to rear.
- print(self.queue[i]) : Displays each queue element.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Queue Overflow occurs when the queue is full.	Check if self.rear == self.size - 1 before inserting an element.
2.	Queue Underflow occurs when the queue is empty.	Check if self.front == -1 or self.front > self.rear before deleting an element.

6b**. AIM:** Write a python program to implement queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach implement using circular linked list.

PROGRAM:

queue using cll.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/queue using cll.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularQueue:
    def __init__(self, size=5):
        self.size = size
        self.front = None
        self.rear = None
        self.count = 0

    def enqueue(self, x):
        if self.count == self.size:
            print("Queue Overflow")
        else:
            new_node = Node(x)

            if self.front is None:
                self.front = new_node
                self.rear = new_node
                new_node.next = new_node
            else:
                new_node.next = self.front
                self.rear.next = new_node
                self.rear = new_node

            self.count += 1
            print(f"{x} inserted into the queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            x = self.front.data

            if self.front == self.rear:
                self.front = None
                self.rear = None
            else:
                self.front = self.front.next
                self.rear.next = self.front
```

Ln: 19 Col: 0

queue using cll.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/queue using cll.py (3.14.0)

File Edit Format Run Options Window Help

```
            self.count -= 1
            print(f"{x} deleted from the queue")

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            temp = self.front
            while True:
                print(temp.data)
                temp = temp.next

                if temp == self.front:
                    break

size = int(input("Enter the size of Queue: "))
q = CircularQueue(size)
while True:
    print("\n----- Queue Menu -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()
```

Ln: 85 Col: 16

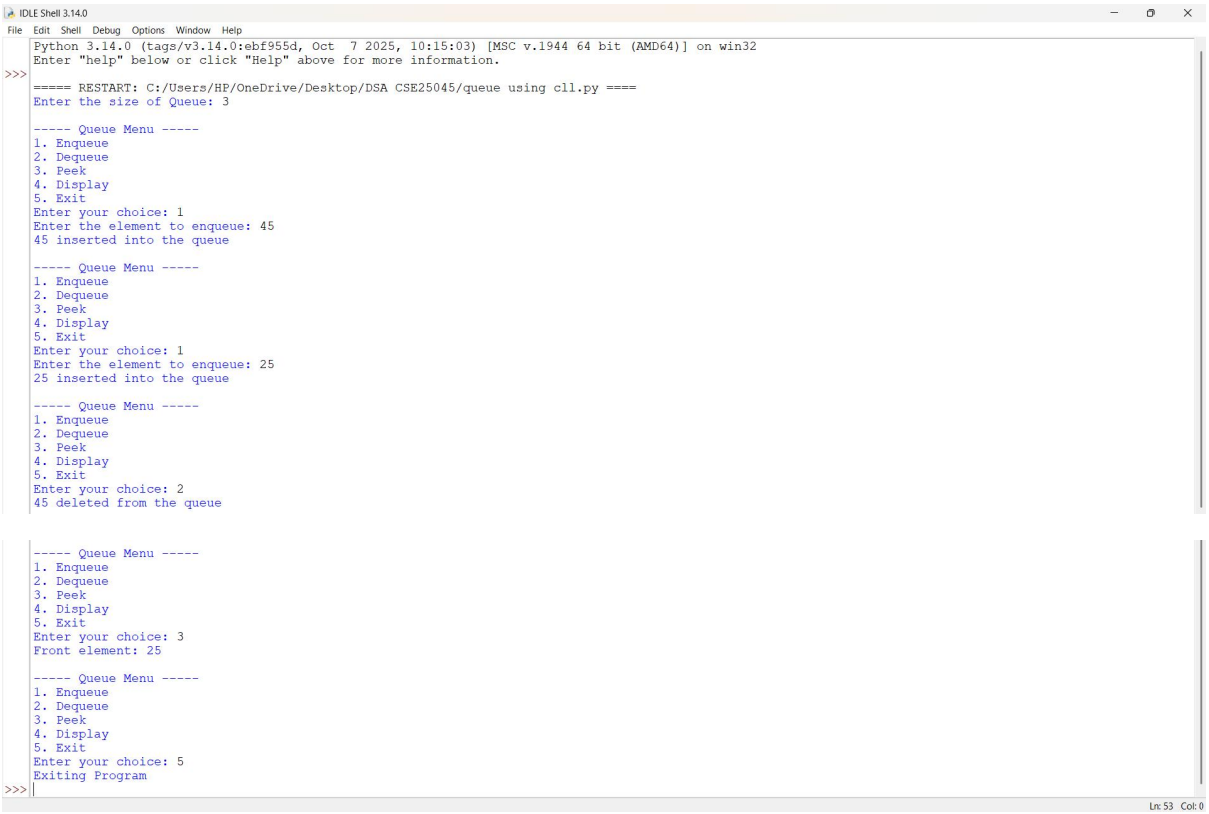
```
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
        print("Invalid Choice!")
```

Ln: 85 Col: 16

OUTPUT:



EXPLANATION:

1. Node Linking

- self.next = None : Initializes the node link as empty.
- new_node = Node(x) : Creates a new node with the given data.
- new_node.next = new_node : Makes the first node point to itself, forming a circular link.

2. Circular Connection During Enqueue

- new_node.next = self.front : Connects the new node to the first node.
- self.rear.next = new_node : Connects the current last node to the new node.
- self.rear = new_node : Updates the last node to the newly inserted node.

3. Circular Connection During Dequeue

- self.front = self.front.next : Moves front to the next node.
- self.rear.next = self.front : Reconnects the last node to the new front node.
- if self.front == self.rear: : Checks whether only one node exists.
- self.front = None : Removes the only node by clearing the front reference.
- self.rear = None : Clears the rear reference.

4. Circular Traversal

- temp = self.front : Starts traversal from the first node.
- temp = temp.next : Moves to the next linked node.
- if temp == self.front: : Detects that traversal has returned to the first node.
- break : Stops traversal after one complete circular cycle.

5. Node Data Access

- self.front.data : Accesses the data stored in the front node.
- temp.data : Accesses the data of the current node during traversal.

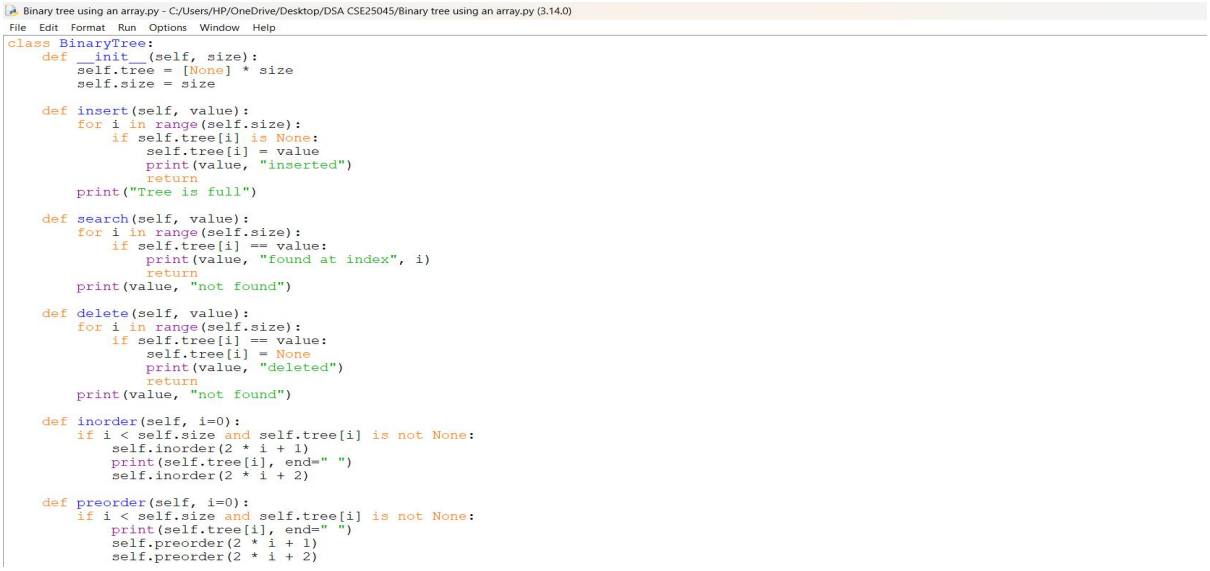
ERROR TABLE:

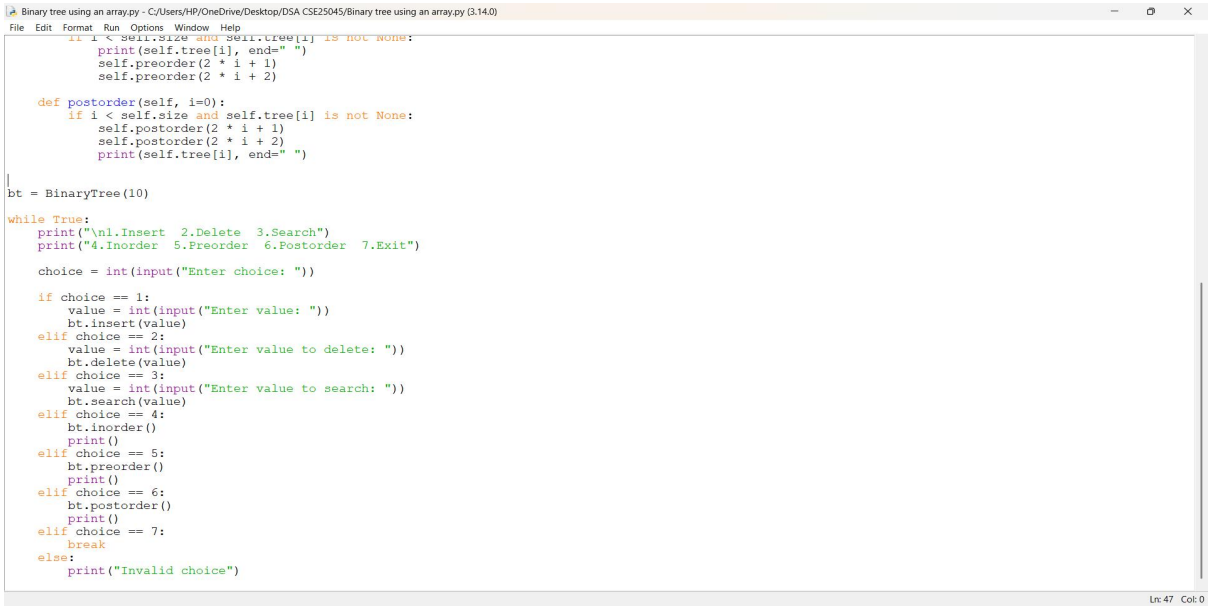
S.NO	Error Obtained	Rectification
1.	new_node.next is not connected to front, breaking the circular link.	Set new_node.next = self.front.
2.	self.rear.next is not updated after insertion or deletion.	Update self.rear.next = new_node during enqueue and self.rear.next = self.front during dequeue.

WEEK – 7

7**. AIM:** Write a python program to implement binary tree using an array and a linked list with all of its operation.

PROGRAM: Using an array





```
Binary tree using an array.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/Binary tree using an array.py (3.14.0)
File  Edit  Format  Run  Options  Window  Help
        if i < self.size and self.tree[i] is not None:
            print(self.tree[i], end=" ")
            self.preorder(2 * i + 1)
            self.preorder(2 * i + 2)

    def postorder(self, i=0):
        if i < self.size and self.tree[i] is not None:
            self.postorder(2 * i + 1)
            self.postorder(2 * i + 2)
            print(self.tree[i], end=" ")

bt = BinaryTree(10)

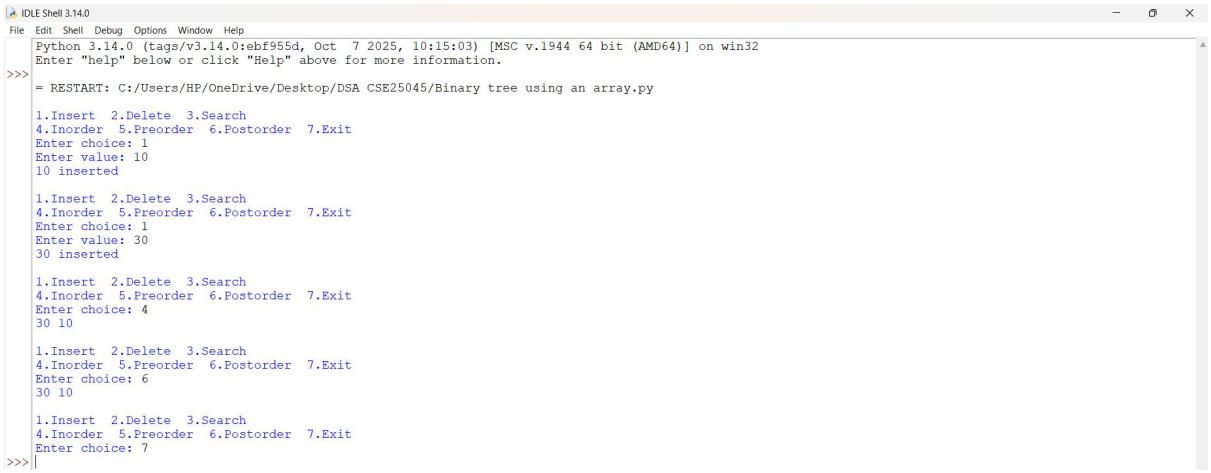
while True:
    print("\n1.Insert  2.Delete  3.Search")
    print("4.Inorder  5.Preorder  6.Postorder  7.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        bt.insert(value)
    elif choice == 2:
        value = int(input("Enter value to delete: "))
        bt.delete(value)
    elif choice == 3:
        value = int(input("Enter value to search: "))
        bt.search(value)
    elif choice == 4:
        bt.inorder()
        print()
    elif choice == 5:
        bt.preorder()
        print()
    elif choice == 6:
        bt.postorder()
        print()
    elif choice == 7:
        break
    else:
        print("Invalid choice")
```

Ln: 47 Col: 0

OUTPUT:



```
IDLE Shell 3.14.0
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/Binary tree using an array.py

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 1
Enter value: 10
10 inserted

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 1
Enter value: 30
30 inserted

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 4
30 10

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 6
30 10

1.Insert  2.Delete  3.Search
4.Inorder  5.Preorder  6.Postorder  7.Exit
Enter choice: 7
>>>
```

PROGRAM: Using linked list



```
*Binary tree using linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/Binary tree using linked list.py (3.14.0)*
File  Edit  Format  Run  Options  Window  Help
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        new_node = Node(data)

        if self.root is None:
            self.root = new_node
            print(data, "inserted")
            return
        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.left is None:
                current.left = new_node
                print(data, "inserted")
                return
            else:
                queue.append(current.left)

            if current.right is None:
                current.right = new_node
                print(data, "inserted")
                return
            else:
                queue.append(current.right)

    def search(self, data):
        if self.root is None:
            print("Tree is empty")
            return

        queue = [self.root]
```

Ln: 120 Col: 0

```
*Binary tree using linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/Binary tree using linked list.py (3.14.0)*
File  Edit  Format  Run  Options  Window  Help
        while queue:
            current = queue.pop(0)

            if current.data == data:
                print(data, "found")
                return

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        print(data, "not found")

    def delete(self, data):
        if self.root is None:
            print("Tree is empty")
            return
        queue = [self.root]
        target = None
        deepest = None
        parent = None
        deepest_parent = None

        while queue:
            current = queue.pop(0)

            if current.data == data:
                target = current

            if current.left:
                parent = current
                queue.append(current.left)
                deepest_parent = current

            if current.right:
                parent = current
                queue.append(current.right)
                deepest_parent = current

            deepest = current
Ln: 120  Col: 0
```

```
*Binary tree using linked list.py - C:/Users/HP/OneDrive/Desktop/DSA CSE25045/Binary tree using linked list.py (3.14.0)*
File  Edit  Format  Run  Options  Window  Help
        if target is None:
            print(data, "not found")
            return

        if deepest == self.root:
            self.root = None
        else:
            if deepest_parent.left == deepest:
                deepest_parent.left = None
            else:
                deepest_parent.right = None
            target.data = deepest.data
        print(data, "deleted")

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

bt = BinaryTree()
while True:
    print("\n1.Insert  2.Delete  3.Search")
    print("4.Inorder  5.Preorder  6.Postorder  7.Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        bt.insert(value)
Ln: 119  Col: 17
```

```
    elif choice == 2:
        value = int(input("Enter value to delete: "))
        bt.delete(value)

    elif choice == 3:
        value = int(input("Enter value to search: "))
        bt.search(value)

    elif choice == 4:
        bt.inorder(bt.root)
        print()

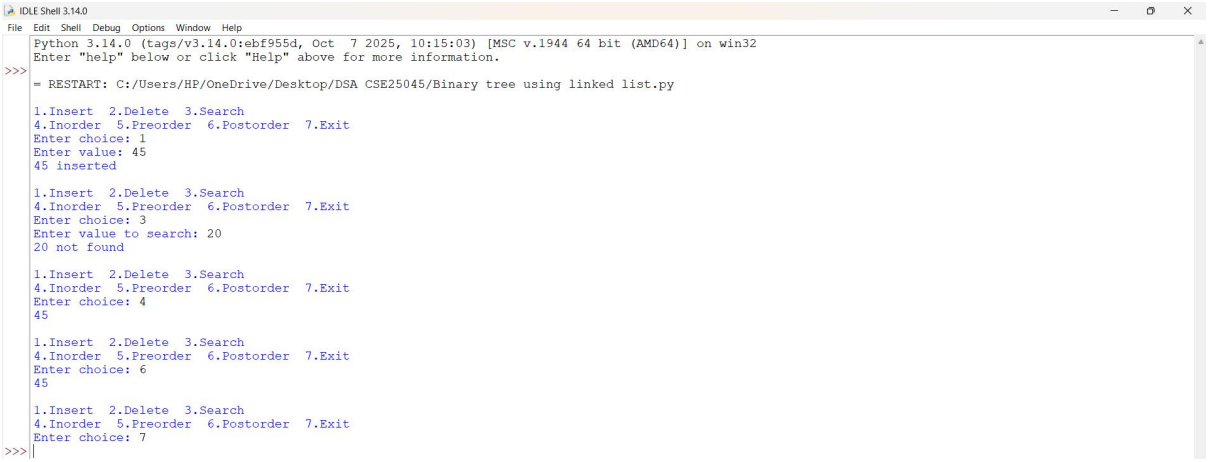
    elif choice == 5:
        bt.preorder(bt.root)
        print()

    elif choice == 6:
        bt.postorder(bt.root)
        print()

    elif choice == 7:
        break

    else:
        print("Invalid choice")
Ln: 119  Col: 17
```

OUTPUT:



EXPLANATION:

Binary Tree Using Array

- class BinaryTree: Defines the binary tree class.
- __init__(self, size): Creates a fixed-size array filled with None.
- insert(value): Inserts a value into the first empty array position.
- search(value): Searches for a value and displays its index.
- delete(value): Deletes a value by replacing its array position with None.
- inorder(i=0): Traverses Left → Root → Right using array indices.
- preorder(i=0): Traverses Root → Left → Right.
- postorder(i=0): Traverses Left → Right → Root.
- bt = BinaryTree(10): Creates an array-based tree with capacity 10.
- Child indices: Left child = 2*i + 1; right child = 2*i + 2.

Binary Tree Using Linked Representation

- class Node: Defines a node containing data and left and right references.
- __init__(self, data): Initializes the node's data and sets both child references to None.
- __init__(self): Initializes an empty tree with self.root = None.
- insert(data): Creates a node and uses a queue to insert it at the first available position in level order.
- search(data): Uses a queue to search nodes level by level.
- delete(data): Replaces the target's data with the deepest node's data and removes the deepest node. The supplied code has a bug in tracking the deepest node's parent.

- inorder(node), preorder(node), postorder(node): Perform the three traversals described in Code 1, using node references instead of array indices.
- bt = BinaryTree(): Creates an empty linked binary tree.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Array-based deletion sets a node to None, making its existing children inaccessible during traversal. Ensure deletion preserves the tree structure or use proper deepest-node replacement.	Ensure deletion preserves the tree structure or use proper deepest-node replacement.
2.	Linked-tree deletion tracks deepest_parent incorrectly.	Update the parent reference whenever a child is visited, and track the actual parent of the deepest node.
3.	parent variable in the linked-tree deletion method is assigned but never used.	Remove the unused variable or use it to track the deepest node's parent correctly.

WEEK – 8

8. **AIM:** Write a python program to implement binary search trees with its operation.

PROGRAM:



```
        elif key > root.data:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.find_min(root.right)
            root.data = temp.data
            root.right = self.delete(root.right, temp.data)

        return root

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")

tree = BST()

while True:
    print("\n\n1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")
```

```
    choice = int(input("Enter your choice: "))
    if choice == 1:
        data = int(input("Enter value to insert: "))
        tree.root = tree.insert(tree.root, data)
        print("Node inserted successfully.")

    elif choice == 2:
        key = int(input("Enter value to search: "))
        result = tree.search(tree.root, key)

        if result:
            print("Element found.")
        else:
            print("Element not found.")

    elif choice == 3:
        key = int(input("Enter value to delete: "))
        if tree.search(tree.root, key):
            tree.root = tree.delete(tree.root, key)
            print("Node deleted successfully.")
        else:
            print("Element not found.")

    elif choice == 4:
        print("Inorder Traversal:", end=" ")
        tree.inorder(tree.root)

    elif choice == 5:
        print("Preorder Traversal:", end=" ")
        tree.preorder(tree.root)

    elif choice == 6:
        print("Postorder Traversal:", end=" ")
        tree.postorder(tree.root)

    elif choice == 7:
        print("Program terminated.")
        break

    else:
        print("Invalid choice. Please try again.")
```

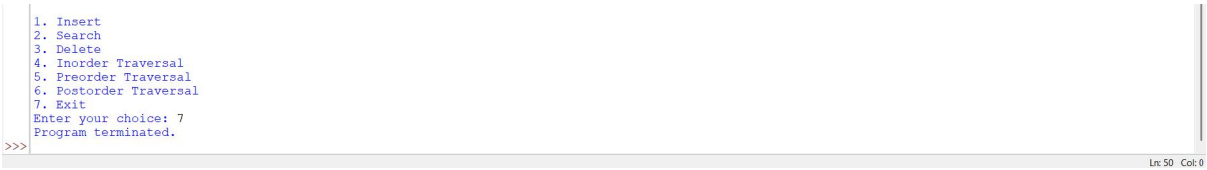
OUTPUT:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> ========== RESTART: C:/Users/HP/OneDrive/Desktop/DSA CSE25045/BST.py ==========


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 87
Node inserted successfully.


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 23
Node inserted successfully.


1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 6
Postorder Traversal: 23 87
```



EXPLANATION:

BST Initialization

- class BST: Defines the BST class.
- self.root = None: Initializes the root.

Insertion

- insert(root, data): Inserts a value according to BST ordering.
- root is None: Creates a new node.
- data < root.data: Recursively inserts into the left subtree.
- data > root.data: Recursively inserts into the right subtree.
- return root: Returns the updated subtree root.

Searching

- search(root, key): Searches recursively for the specified key.
- root is None or root.data == key: Returns the node or None.
- key < root.data: Searches the left subtree; otherwise, searches the right subtree.

Finding Minimum

- find_min(root): Moves left until the leftmost node is reached.
- return root: Returns the minimum-valued node.

Deletion

- delete(root, key): Deletes a node while preserving BST ordering.
- Handles nodes with zero or one child by returning the appropriate child.
- For two children, finds the inorder successor, copies its value, and deletes the successor.

Main Program

- tree = BST(): Creates a BST object.
- tree.root = tree.insert(...) and tree.root = tree.delete(...): Update the root after modifications.

ERROR TABLE:

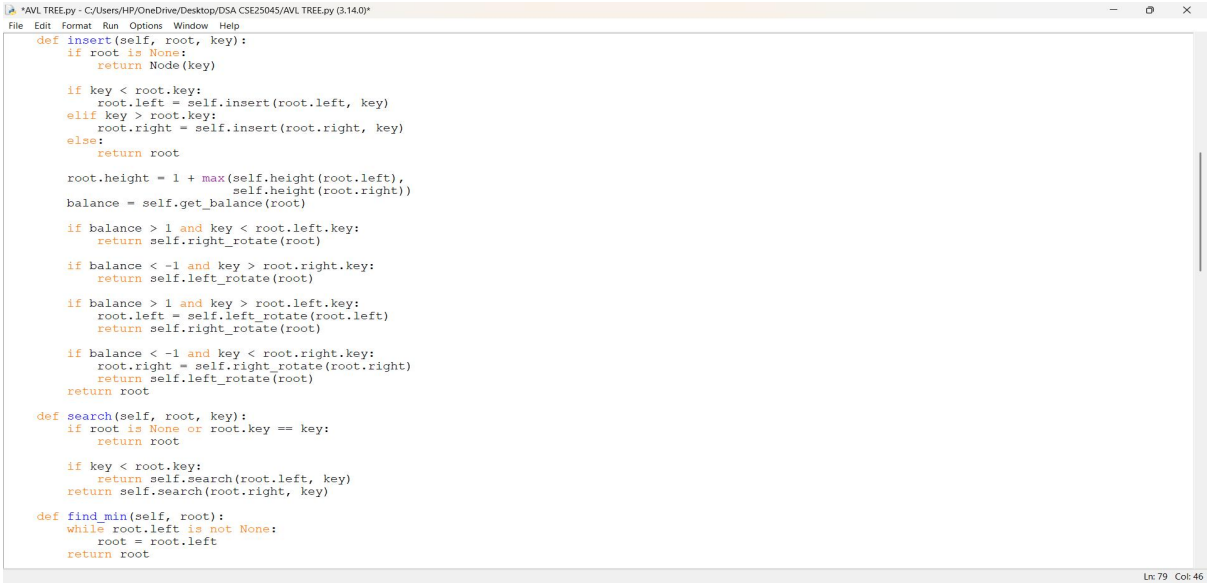
S.NO	Error Obtained	Rectification
1.	Incorrect root assignment after insertion or deletion may cause the updated tree structure to be lost.	Assign the returned node to tree.root using tree.root = tree.insert(tree.root, data) and tree.root = tree.delete(tree.root, key).
2.	Deleting a node with two children may violate BST ordering if the replacement node is not selected correctly.	Use find_min(root.right) to find the inorder successor and delete that successor from the right subtree.

WEEK – 9

9. **AIM:** Write a python program to implement AVL trees with its operation.

PROGRAM:





```
    def delete(self, root, key):
        if root is None:
            return root

        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.find_min(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)

        if root is None:
            return root

        root.height = 1 + max(self.height(root.left),
                              self.height(root.right))

        balance = self.get_balance(root)

        if balance > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)

        if balance > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)

        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)

        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)

        return root
```

```
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")

tree = AVLTree()
root = None
while True:
    print("\n\n1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        key = int(input("Enter value to insert: "))
        root = tree.insert(root, key)
        print("Node inserted successfully.")

    elif choice == 2:
        key = int(input("Enter value to search: "))
        if tree.search(root, key):
            print("Element found.")
        else:
            print("Element not found.")
```

```
    elif choice == 3:
        key = int(input("Enter value to delete: "))
        if tree.search(root, key):
            root = tree.delete(root, key)
            print("Node deleted successfully.")
        else:
            print("Element not found.")

    elif choice == 4:
        print("Inorder Traversal:", end=" ")
        tree.inorder(root)

    elif choice == 5:
        print("Preorder Traversal:", end=" ")
        tree.preorder(root)

    elif choice == 6:
        print("Postorder Traversal:", end=" ")
        tree.postorder(root)

    elif choice == 7:
        print("Program terminated.")
        break

    else:
        print("Invalid choice.")
```

OUTPUT:



EXPLANATION:

1. Node Class

- self.height = 1: Stores the height of each node, which is required for balancing.

2. Height Calculation

- height(root): Returns the height of a node; returns 0 for None.
- get_balance(root): Calculates the balance factor as height(root.left) - height(root.right).

3. Rotations

- right_rotate(y): Performs a right rotation to rebalance a subtree.
- left_rotate(x): Performs a left rotation to rebalance a subtree.
- T2: Temporarily stores a subtree during rotation.
- Node heights are recalculated after each rotation.
- The rotations handle LL, RR, LR, and RL imbalance cases.

4. Insertion

- root.height = 1 + max(...): Updates the height after insertion.
- balance = self.get_balance(root): Checks whether the subtree is unbalanced.

- The four conditional statements identify the LL, RR, LR, and RL cases and apply the required rotations.

5. Deletion

- Recalculates node heights after deletion.
- Checks the updated balance factor.
- Uses the child node's balance factor to select the appropriate single or double rotation.
- Returns the new subtree root after rebalancing.

6. Main Program

- root = tree.insert(root, key): Updates the root after insertion and rebalancing.
- root = tree.delete(root, key): Updates the root after deletion and rebalancing.

ERROR TABLE:

S.NO	Error Obtained	Rectification
1.	Incorrect height update after insertion or deletion may cause an incorrect balance factor.	Update the node height using 1 + max(height(left), height(right)) after modifying the subtree.
2.	Incorrect rotation selection can leave the AVL tree unbalanced.	Check the balance factor and apply the correct LL, RR, LR, or RL rotation during insertion and deletion.